



Ducks in a Row

Certificate Lifecycle Automation & Management for ADCS

Installation & Configuration Guide

Make ADCS work like Let's Encrypt for your internal network.

Version 1.0.0
2026

Table of Contents

1. Quick Start	3
2. System Requirements	15
3. Installation	18
4. Configuration	22
5. ADCS Configuration	29
6. Verifying Your Installation	32
7. Connecting ACME Clients	37
8. Kubernetes and OpenShift	44
9. External Account Binding	53
10. Device Attestation	56
11. Revocation	61
12. Expiry Alerts	64
13. Hardening	68
14. Backup and Restore	72
15. Troubleshooting	74
16. Uninstalling	86
17. Glossary	88

1. Quick Start

Get from a fresh install to your first issued certificate. Plan for about fifteen minutes the first time, most of it spent on Active Directory permissions.

New to ACME or ADCS? The glossary explains the key terms.

Before you start

You need:

- A Windows Server that is **domain joined** to the same forest as your ADCS certificate authority.
- The **ADCS Remote Administration Tools** installed on that server. They register the COM classes Ducks in a Row uses to talk to the CA:

```
Install-WindowsFeature RSAT-ADCS-Mgmt
```

- The server's **machine account** granted, on the CA:
 - **Enroll** on each certificate template you want to expose, and
 - **Read** on the CA itself (Certification Authority console, CA properties, Security tab). Read is what lets the dashboard sync the certificate inventory. Without it, enrolment still works but the dashboard stays empty.
- Optional: your CA connection string in CAHOST\CA Name form. The setup wizard discovers the CAs published in Active Directory on its own; you only need the string for manual entry. Find it with:

```
certutil -config - -ping
```

1. Install

Download from the [latest release](#) and run it on the server. There are two downloads and you almost certainly want the first:

- `Ducks-in-a-Row-Setup.exe` carries the ASP.NET Core 10 runtime inside it, so it installs on a server with no internet access from the one file. It skips the runtime install when one is already present, and never removes it on uninstall.
- `Ducks-in-a-Row.msi` is the application alone, for deployment systems that manage the runtime themselves. Without the runtime the service fails before it can log anything and the installer reports a misleading **Error 1920**.

Both installers are code signed by Haruspex Systems B.V. Check the signature and the SHA256 checksum on the release page before you run either one, as [checking the download](#) describes. Windows SmartScreen may still warn about a new release; if it does, **More info** should name Haruspex Systems B.V. as the publisher. The install wizard lets you:

- choose the destination folder (default `C:\Program Files\Ducks in a Row\`),
- choose the data folder for the database, logs, and runtime configuration (default `C:\ProgramData\Ducks in a Row\`),

- choose whether to start the service immediately,
- open the setup page in your browser from the completion screen.

In all cases the installer:

- copies the application to the destination folder you chose,
- creates the data folder and records its location in the registry so the service finds it,
- registers the Windows service **DucksInARow** (display name "Ducks in a Row Certificate Proxy", runs as LocalSystem, set to start automatically on boot),
- opens inbound firewall rules on **TCP 5000** (HTTP) and **TCP 5001** (HTTPS),
- adds a **Ducks in a Row** Start Menu shortcut to the web UI.

Unattended install

```
msiexec /i Ducks-in-a-Row.msi /qn INSTALLFOLDER="D:\Ducks in a Row" DATAFOLDER="D:\DucksData" START_SERVICE=0
```

- **INSTALLFOLDER**: destination folder (optional; defaults to C:\Program Files\Ducks in a Row).
- **DATAFOLDER**: data folder (optional; defaults to C:\ProgramData\Ducks in a Row). Choose it at install time; moving it on a later upgrade is not supported.
- **START_SERVICE**: 1 (default) starts the service after install; 0 registers it but leaves it stopped until the next boot or a manual start.

The "open the setup page" prompt only appears in the interactive wizard, so a /qn install never launches a browser.

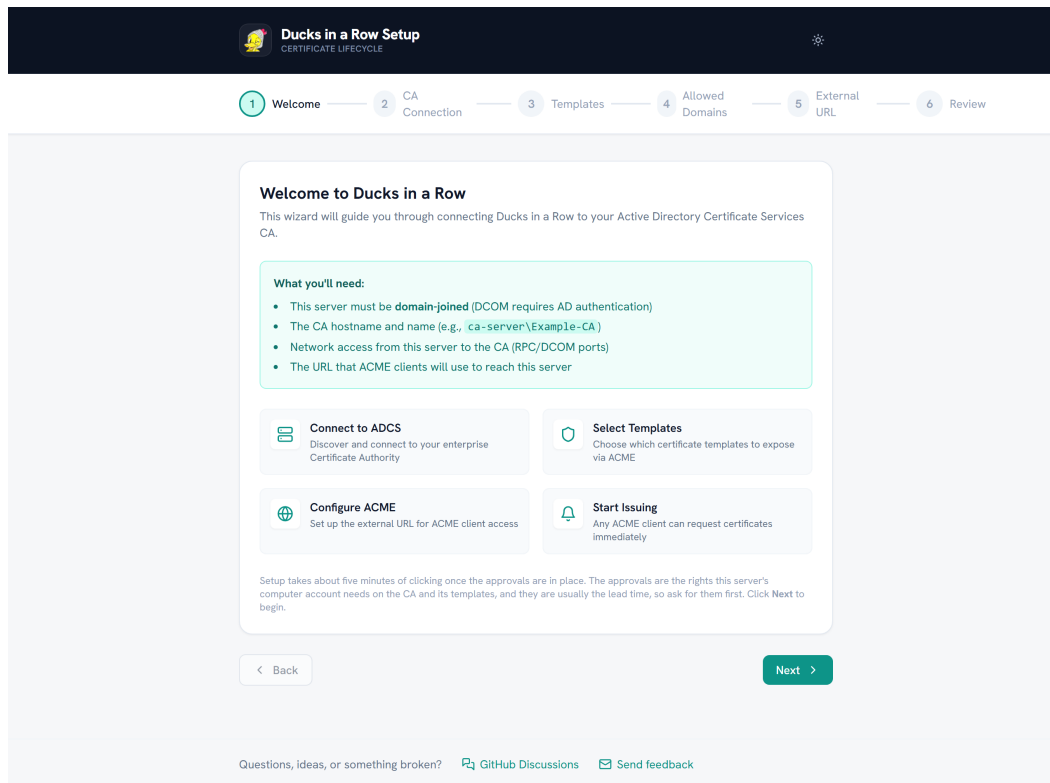
2. Run the setup wizard

Open the dashboard in a browser on <https://your-server:5001> (or <http://your-server:5000>). You must sign in as a member of the administrator group. By default that is the server's built in Administrators group; set `Auth:AdminGroup` in `settings.json` in the data folder to delegate to a specific Windows or Active Directory group.

The HTTPS endpoint uses a self signed certificate out of the box (generated in the data folder as `ducks-selfsigned.pfx`), so your browser will warn on first visit. For production, give Ducks in a Row a certificate your clients already trust (see [Giving clients a TLS certificate they trust](#) below).

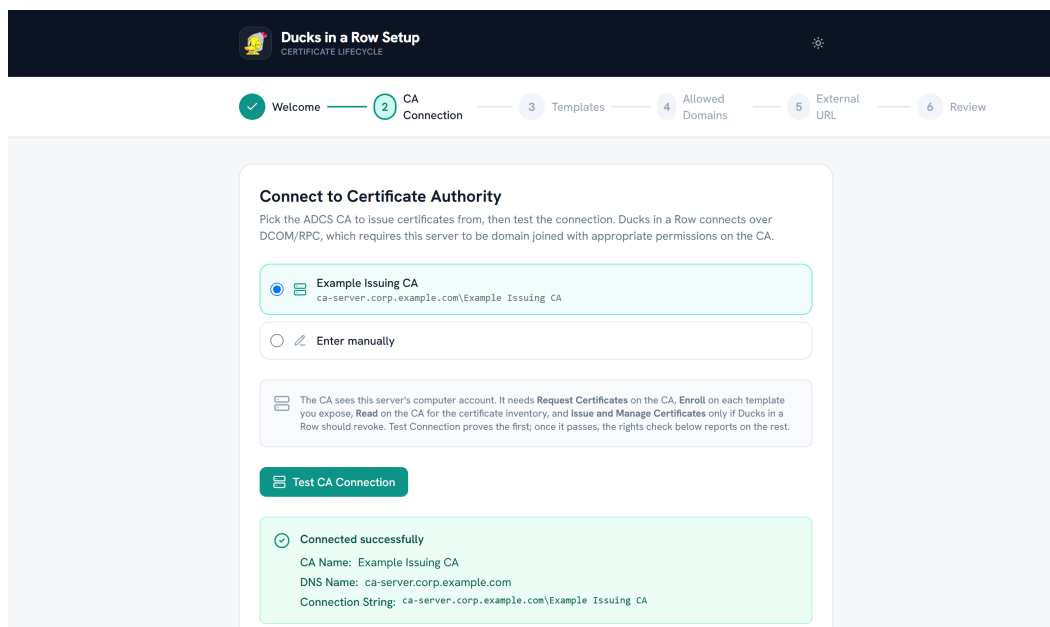
Until setup completes the service is **unconfigured**: it serves the wizard and the dashboard shell, and every CA operation answers 503. It never falls back to a fake CA on its own.

On first run you are taken to the setup wizard. It walks you through six short steps. It remembers where you were, so a service restart in the middle of it does not send you back to the beginning.



Setup wizard welcome step listing what you need before you start

1. **Welcome.** A short checklist of what you need before you start.
2. **Connect a CA.** The wizard discovers the CAs published in Active Directory and lets you pick one, or enter `CAHOST\CA` Name by hand. It then tests the connection through the same DCOM path the service uses in production. A pass proves that the service's account holds Request Certificates on the CA, and the wizard then checks the service's other rights. The Review step runs the full check, template permissions included, and says plainly what is not proven yet. See [the service's rights](#).



Setup wizard connection step showing a discovered CA and the test connection button

3. **Choose a template.** It lists the templates that CA publishes, checks each one for ACME readiness, and lets you choose the template to expose. No template is exposed over ACME until a selection is recorded on this step: on a fresh install, every template's directory URL answers 403 until then.

Ducks in a Row Setup
CERTIFICATE LIFECYCLE

Progress: 1 Welcome 2 CA Connection 3 **Templates** 4 Allowed Domains 5 External URL 6 Review

Select a Certificate Template

Choose the AD CS template to expose via ACME. Only templates that can issue server authentication certificates are listed. The template gets its own ACME directory URL: `/acme/{template}/directory`

⚠ EKU could not be verified
This host may not be domain joined, or the service account cannot read the AD Configuration partition. Showing all templates. Confirm your choice issues server authentication certificates.

- ☒ **Web Server**
Name: WebServer OID: 1.3.6.1.4.1.311.21.8.1 /acme/WebServer/directory
- ☐ **Computer**
Name: Machine OID: 1.3.6.1.4.1.311.21.8.2
- ☐ **User**
Name: User OID: 1.3.6.1.4.1.311.21.8.3
- ☐ **Code Signing**
Name: CodeSigning OID: 1.3.6.1.4.1.311.21.8.4
- ☐ **Web Server EC**
Name: WebServerEC OID: 1.3.6.1.4.1.311.21.8.5
- ☐ **Web Server EC (encryption purpose)**
Name: WebServerECDH OID: 1.3.6.1.4.1.311.21.8.6

ACME readiness

- ☐ Issues server authentication certificates
The EKU set could not be read from AD.
- ☒ No CA manager approval required
- ☒ No enrollment agent signatures required
- ☒ Subject and SAN are taken from the request
- ☐ Requires RSA keys, minimum 2048 bits
ACME clients must request a matching key type or the CA denies the request.
- ☒ The display name can be used in the directory URL

[Questions, ideas, or something broken?](#) [GitHub Discussions](#) [Send feedback](#)

Setup wizard templates step showing the ACME readiness checklist

4. **Allowed domains.** Choose which domains this server may issue certificates for. On a domain joined server the wizard arrives here with the restriction already on and your Active Directory domain filled in. An entry covers the domain and all of its subdomains, so `corp.example.com` also covers `web.corp.example.com`; wildcard entries are not needed and are not accepted. You can turn the restriction off, but leaving it on is the recommendation. It restricts issuance **through Ducks in a Row only**: the CA itself can still issue for any name through its own tools, so this complements CA side controls like name constraints rather than replacing them.

Ducks in a Row Setup
CERTIFICATE LIFECYCLE

Progress: Welcome — CA Connection — Templates — **4 Allowed Domains** — 5 External URL — 6 Review

Allowed Domains

Choose which domains this server may issue certificates for.

☒ **Restrict certificate issuance to these domains (recommended)**
When off, ACME clients can order a certificate for any name that passes challenge validation, including names outside your organization.

corp.example.com + Add

corp.example.com includes subdomains ×

An entry covers the domain and all of its subdomains: corp.example.com also allows web.corp.example.com, so wildcard entries are not needed. This restricts issuance through Ducks in a Row only; the certificate authority itself can still issue for any name through its own tools. You can change the list later on the Settings page, and changes apply without a restart.

< Back Next >

Questions, ideas, or something broken? [GitHub Discussions](#) [Send feedback](#)

Setup wizard allowed domains step with the restriction enabled and one domain listed

5. **Set the external URL.** This is the address clients will use to reach the server. It comes prefilled with a suggestion and is checked for you. On this step the wizard can also enrol an HTTPS certificate for the server from your CA in one click, so clients trust the connection (see [Giving clients a TLS certificate they trust](#)).

Ducks in a Row Setup
CERTIFICATE LIFECYCLE

Progress: Welcome — CA Connection — Templates — Allowed Domains — **5 External URL** — 6 Review

External URL

This is the base URL that ACME clients will use to reach the Ducks in a Row server. It must be reachable from every machine that needs to request certificates.

ACME clients will construct directory URLs like:

```
https://certus.corp.example.com:5001/acme/webServer/directory
```

Use HTTPS for production, and include the port. The service listens on 5001 for HTTPS (5000 for HTTP) instead of 443, because IIS and the ADCS web roles often occupy 443 on Windows servers and a port collision would stop the service from starting. The ports can be changed under Kestrel:Endpoints in appsettings.json; update the firewall rules to match.

Server URL

Validate

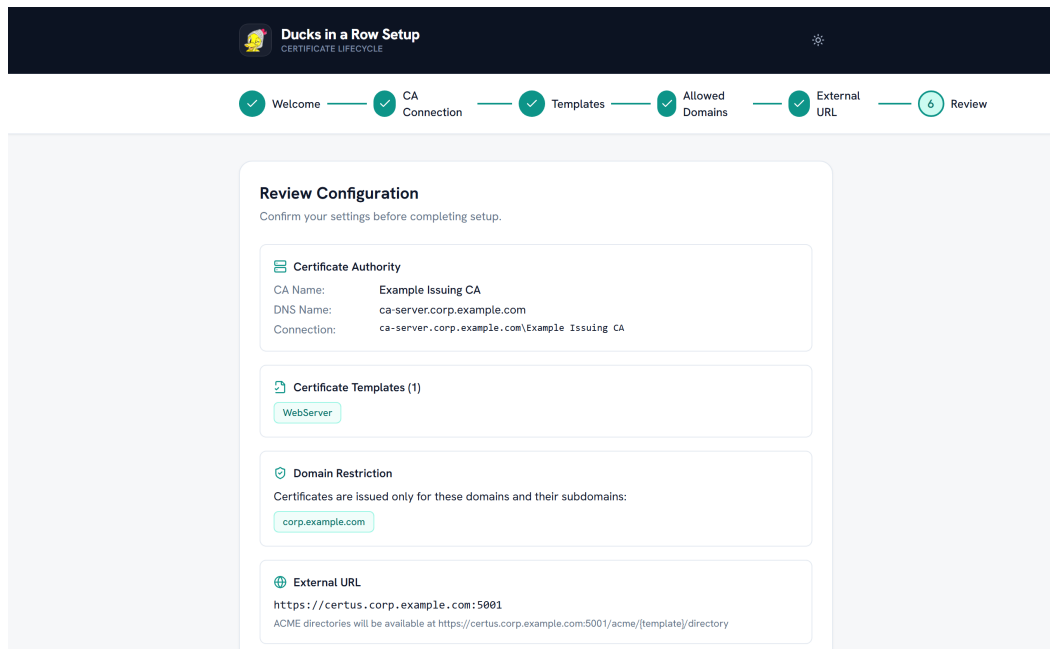
URL accepted
Reachability was not checked in demo mode. ACME clients should target:
`https://certus.corp.example.com:5001/acme/{template}/directory`

[< Back](#) [Next >](#)

[Questions, ideas, or something broken?](#) [GitHub Discussions](#) [Send feedback](#)

Setup wizard external URL step showing the server URL and the directory URL clients will use

- Review and apply.** The step summarises your choices and runs the rights check again, with a row for each template you chose. Unless step 5 enrolled an HTTPS certificate for the server, Complete Setup waits for you to tick a box naming the rights that are not proven yet. It never refuses because a right is missing. Your choices are then saved to `settings.json` in the data folder and the service restarts itself to load them. The wizard waits for the service to come back, then shows a ready to run certbot command and opens the dashboard.



Setup wizard review step summarising the CA, the template, the allowed domains and the external URL

To point an existing install at a different CA later, you have two options. Either edit `Certus:CaConnectionString` in `settings.json` in the data folder and restart the service, or clear that value and restart to reopen the wizard. Setup locks once it is complete, so the wizard will not reopen while a CA is still configured.

3. Request your first certificate

Each certificate template is its own ACME endpoint:

```
https://your-server:5001/acme/<template>/directory
```

`<template>` is the template's programmatic name (its AD cn) or its display name. Display names with spaces work; the client URL encodes them. A display name carrying an invisible character, such as a soft hyphen left behind by a paste from a word processor, is refused with a 400; use the programmatic name and see [Troubleshooting](#).

Point any standard ACME client at that directory URL and request a certificate. A quick test with certbot:

```
certbot certonly --standalone \
  --server https://your-server:5001/acme/WebServer/directory \
  --email you@example.com \
  -d host.corp.example.com \
  --key-type rsa --rsa-key-size 2048
```

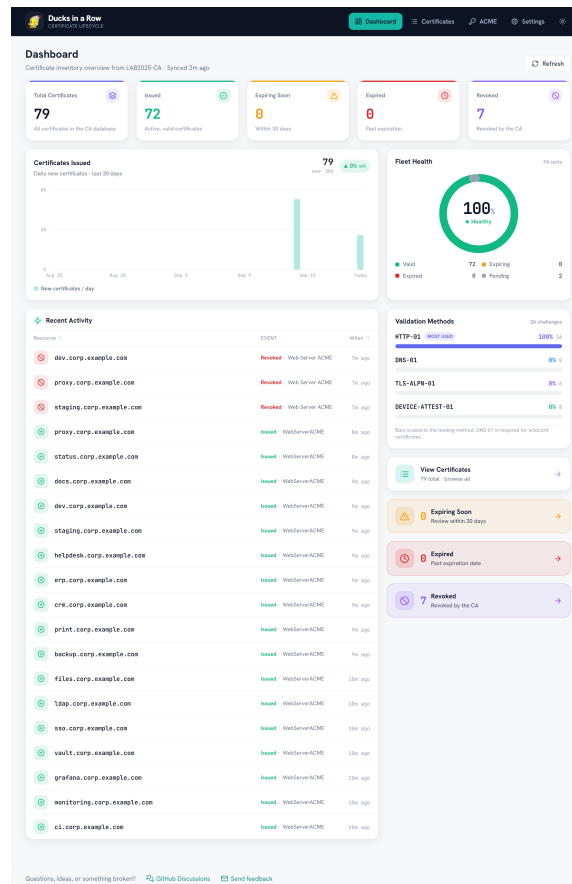
Match the key type to the template. Your client must ask for the key type the template wants, and templates differ. An RSA template needs an RSA key; an ECDSA_P256 template needs a key on that curve. Ask for the wrong one and the CA policy module refuses at finalize with `Denied by Policy Module`. You do not have to work this out yourself. The wizard's template step reports the key algorithm it read from the template, and the **Client setup** snippets on the dashboard's ACME page arrive with the right flags already filled in. Copy them from there rather than from memory. The stock `Web Server` ACME template is RSA, which is why the command above forces RSA: most clients default to an

elliptic curve key. See [connecting ACME clients](#) for the per client flags.

Out of the box no account pre-registration is needed and no external account binding is required: the client creates an account from its own key the first time it connects. If an administrator has set EAB enforcement to Optional or Required, the client needs a credential as well. See [external account binding](#).

See [Connecting ACME clients](#) for certbot, win-acme, Caddy, Traefik, and Posh-ACME.

4. Confirm it worked



The dashboard showing the certificate inventory and expiry tiles after the first sync

- The client writes out the issued certificate and chain.
- The certificate appears in the dashboard inventory within one sync cycle (every 5 minutes by default, Certus:SyncIntervalMinutes).
- The health endpoint reports healthy:

```
Invoke-RestMethod https://your-server:5001/health
```

If anything misbehaves, see [Troubleshooting](#).

Configuration reference

The settings below are the ones a first install usually touches. [Configuration](#) is the complete list, including alerting, rate limiting, challenge egress and the keys that ship as an explicit null.

Configuration is layered. Shipped defaults live in `appsettings.json` in the installation folder; the setup wizard writes instance settings (CA connection string, external URL) to `settings.json` in the **data folder**, which overrides them and survives upgrades. Environment variables override both. Restart the service after editing either file.

Setting	Default	Purpose
<code>Certus:CaConnectionString</code>	null (unconfigured; the wizard sets it)	ADCS CA in CAHOST\CA Name form
<code>Certus:DatabasePath</code>	<code>ducks.db</code> in the data folder	SQLite database file
<code>Certus:ExternalUrl</code>	null	The base URL you expect clients to use. Recorded and checked at startup; ACME URLs themselves follow the host the client connects to
<code>Certus:SyncIntervalMinutes</code>	5	How often the dashboard syncs from the CA
<code>Certus:RequestHistoryDays</code>	30	How far back the sync reaches for pending, denied, and failed requests, so their detail pages can show the CA's own explanation. Issued and revoked certificates are always synced in full. Set to 0 to skip those three passes entirely, which also empties the Pending, Denied, and Failed filters on the certificate list
<code>Certus:Acme:ExposeAllTemplates</code>	false	Break glass override: expose every CA published template over ACME, ignoring the wizard's template selection. Leave it false; the open posture is logged as a warning at startup
<code>Certus:EnableWalMode</code>	true	SQLite write ahead logging, for better concurrent read and write performance. Set it to false on a filesystem that cannot support WAL, or when a backup or replication tool needs a single database file with no -wal and -shm sidecars. Applied in both directions on every start, so changing it converts the existing database. See Troubleshooting before moving the data folder to a network share

Setting	Default	Purpose
Auth:Mode	Negotiate	Windows Integrated Authentication for the dashboard
Auth:AdminGroup	null (built in Administrators)	Group allowed into the dashboard and setup
Auth:RequireHttps	true	HSTS and HTTP to HTTPS redirect outside development
Auth:TrustedProxies	[]	Reverse proxy IPs whose X-Forwarded-* headers are trusted

Challenge validation egress

For HTTP-01 and TLS-ALPN-01, the server itself connects out to the host being validated. Loopback, link local (including the cloud metadata address), and IPv6 unique local targets are always refused; the RFC 1918 private ranges are allowed by default, because an internal CA usually issues for exactly those addresses. If everything you validate is public, set

Certus:Acme:ChallengeValidation:BlockPrivateRanges to true in settings.json. The [hardening guide](#) explains the tradeoff and the rest of the egress settings.

Giving clients a TLS certificate they trust

ACME clients refuse to connect to an ACME server whose own TLS certificate they do not trust. The easiest fix is built into the app: on the external URL step, the setup wizard can enrol an HTTPS certificate for this server from the CA you connected, install it, and reload the service to serve it. The **Settings** page can renew that certificate later. Try these built in options first.

If you would rather supply your own certificate (for example, one from a different issuer), point Kestrel at a PFX file instead:

```
"Kestrel": {
  "Endpoints": {
    "Https": {
      "Url": "https://0.0.0.0:5001",
      "Certificate": {
        "Path": "C:\\ProgramData\\Ducks in a Row\\ducks.pfx",
        "Password": "your-pfx-password"
      }
    }
  }
}
```

For a quick lab test only, you can set Auth:RequireHttps to false and use plain HTTP on port 5000 instead.

See the Alert Configuration section of the Installation Guide for SMTP and webhook expiry notifications, or the [Troubleshooting](#) page for common issues.

2. System Requirements

What the server needs before you install. If you only want to get going, the [quickstart](#) covers the short version; this page is the full list, including the network ports and the reason each one is open.

Server requirements

Requirement	Details
Operating system	Windows Server. Verified on Server 2019 and Server 2025. See the Server 2022 note below
Runtime	ASP.NET Core 10.0. Ducks-in-a-Row-Setup.exe installs it for you, offline. Install the Hosting Bundle by hand only if you use the bare MSI
Windows feature	RSAT-ADCS-Mgmt, the ADCS Remote Administration Tools
Processor	Two x64 cores minimum. The work is mostly waiting on input and output, being COM and RPC calls to the CA plus SQLite, with short bursts of certificate cryptography
Memory	1 GB minimum, 2 GB recommended
Disk	About 500 MB for the application and the .NET runtime. Plan at least 5 GB free for database growth, logs and upgrades. At roughly 1000 ACME accounts renewing over three years the database reaches the low hundreds of MB, and log files are capped at 30 daily files
Network	TCP 5000 for HTTP and TCP 5001 for HTTPS, both configurable
Domain	Must be domain joined, because the CA is reached over DCOM

Server 2022 needs current updates first. The .NET 10 runtime requires Control-flow Enforcement Technology on Server 2022, and an installation that has not been serviced since early 2022 does not provide it. Build 20348.587 is confirmed too old; check yours with `winver`. The service then crashes before it can write a log line, and the installer reports the misleading **Error 1920**. Server 2019 predates the requirement and Server 2025 ships with it, so neither is affected.

Installing the prerequisites

Run these in an elevated PowerShell session on the Ducks in a Row server before you install:

```
# Install the ADCS Remote Administration Tools
Install-WindowsFeature RSAT-ADCS-Mgmt

# Verify the feature is installed
Get-WindowsFeature RSAT-ADCS-Mgmt
```

```
# Verify the .NET 10 runtime is present, after the setup bundle
# or a manual Hosting Bundle install
dotnet --list-runtimes
```

Why RSAT-ADCS-Mgmt is required. Ducks in a Row talks to the certification authority through the ADCS COM classes, and it is the Remote Server Administration Tools feature that registers those classes on the system. Without it the COM objects cannot be created at all, and the service fails with `DISP_E_MEMBERNOTFOUND (0x80020003)` during certificate sync and CA queries. See [troubleshooting](#) if you are reading that error now.

ADCS requirements

To issue certificates through ADCS, all of the following must be in place:

- An Active Directory Certificate Services enterprise certification authority.
- The ADCS Remote Administration Tools (RSAT-ADCS-Mgmt) installed on the Ducks in a Row server, as above.
- DCOM and RPC network access from the Ducks in a Row server to the CA server.
- The server's machine account granted **Enroll** on each certificate template you want to expose over ACME.
- The server's machine account granted **Read** on the CA itself. Without it enrolment still works, but the dashboard inventory stays empty.
- Certificate templates published to Active Directory.

The service runs as LocalSystem, so the account that needs those rights is the server's machine account, written `DOMAIN\SERVERNAME$`. Revocation from the dashboard additionally needs **Issue and Manage Certificates** on the CA; see [revocation](#) for what that does and does not allow.

Network ports

Direction	Port	Protocol	Purpose
Inbound to the server	5000, 5001	TCP	ACME clients and dashboard browsers
Outbound to the CA	135	TCP	DCOM and RPC endpoint mapper
Outbound to the CA	49152-65535	TCP	DCOM and RPC dynamic ports
Outbound to targets	80	TCP	HTTP-01 challenge validation
Outbound to DNS	53	TCP and UDP	DNS-01 challenge validation
Outbound to targets	443	TCP	TLS-ALPN-01 challenge validation

The three outbound challenge rows are needed only for the challenge types you actually use. The server is the party that performs validation, so it needs to reach the name being validated, not the other way round.

The [hardening guide](#) covers narrowing that egress.

device-attest-01 needs no outbound rule at all. The device presents its attestation in the challenge response and the server verifies it against a trust anchor it already holds, so nothing is fetched. See [device attestation](#).

3. Installation

Two ways to install. Use the setup bundle unless you have a reason not to.

Installer	What it does	When to use it
Ducks-in-a-Row-Setup.exe	Installs the ASP.NET Core 10 runtime if it is missing, then runs the MSI	Almost always, including on servers with no internet access
Ducks-in-a-Row.msi	The application only. Fails to start if the runtime is absent	Deployment systems that manage the runtime themselves

The setup bundle carries the runtime inside it, so an offline server installs from the one file. It detects an existing ASP.NET Core 10 runtime and skips the install when one is present, and it never removes the runtime on uninstall, because other applications on the server may be using it.

The bare MSI does not install the runtime. Without it the service fails before any managed code runs, and the installer reports **Error 1920**, which almost never means what it says. See [troubleshooting](#).

Check the download

Both installers are signed by Haruspex Systems B.V. and timestamped. Check a download before you run it. The same commands work on the MSI.

```
$signature = Get-AuthenticodeSignature .\Ducks-in-a-Row-Setup.exe
$signature.Status # Valid
$signature.SignerCertificate.Subject # CN=Haruspex Systems B.V., O=Haruspex Systems B.V., ...
$signature.TimeStamperCertificate.Subject # a Certum timestamp authority
```

The status must be `Valid`, the signer must be Haruspex Systems B.V., and the timestamp must be there: it is what keeps the signature valid after the signing certificate itself expires. Anything else, such as `NotSigned`, `HashMismatch`, or a different signer, means the file is not the one we published. Do not run it.

Compare the checksum too, against the SHA256SUMS file on the release page:

```
Get-FileHash .\Ducks-in-a-Row-Setup.exe -Algorithm SHA256
```

When you run a signed installer, the Windows permission prompt names Haruspex Systems B.V. as the verified publisher. Windows SmartScreen may still warn about a new release while the signing certificate builds up a reputation with Microsoft. If it does, **More info** should name the same publisher. If it names anyone else, or an unknown publisher, do not run the file.

Every `.exe` and `.dll` the installer puts in the application folder carries the same signature, so an App Control for Business (WDAC) or AppLocker policy can allow everything it installs with one publisher rule. The setup program itself also runs a few helpers from a temporary folder while it works; those are signed by their own publishers, the WiX Toolset and Microsoft.

Releases up to and including 0.10.0-beta.1 were not signed. For those, the checksum is the only check.

What the installer does

Action	Details
Install the application	C:\Program Files\Ducks in a Row\, or the folder you choose
Create the data folder	C:\ProgramData\Ducks in a Row\, or the folder you choose. Its location is recorded in the registry so the service can find it
Create the log folder	Logs\ inside the data folder
Register the service	DucksInARow, display name "Ducks in a Row Certificate Proxy", starts automatically, runs as LocalSystem
Start the service	Immediately after install, unless you clear the option
Open firewall rules	Inbound TCP 5000 and TCP 5001
Add a Start Menu shortcut	Opens the web interface

The data folder is not removed when you uninstall. See [uninstalling](#).

The install wizard

Running either installer interactively gives you four choices:

1. The destination folder, default C:\Program Files\Ducks in a Row\.
2. The data folder for the database, logs and runtime configuration, default C:\ProgramData\Ducks in a Row\.
3. Whether to start the service immediately.
4. Whether to open the setup page in your browser from the final screen.

Choose the data folder now. Moving it during a later upgrade is not supported.

Unattended install

```
msiexec /i Ducks-in-a-Row.msi /qn `
  INSTALLFOLDER="D:\Ducks in a Row" `
  DATAFOLDER="D:\DucksData" `
  START_SERVICE=0 `
  /l*v "C:\temp\ducks-install.log"
```

Property	Default	Meaning
INSTALLFOLDER	C:\Program Files\Ducks in a Row	Where the application goes
DATAFOLDER	C:\ProgramData\Ducks in a Row	Where the database, logs and settings go
START_SERVICE	1	0 registers the service but leaves it stopped until the next boot or a manual start

The prompt to open the setup page appears only in the interactive wizard, so a `/qn` install never launches a browser.

The setup bundle accepts the same silent switches through its own command line:

```
Ducks-in-a-Row-Setup.exe /quiet /log "C:\temp\ducks-bundle.log"
```

If the service stops

The installer deliberately does not configure automatic service recovery. A service that stops stays stopped until something starts it.

The reason is what this service does when it fails. Almost every way it stops is a fault it found while starting, and usually that is a configuration it cannot work with. It writes the reason to the log, sets a nonzero exit code and stops. A restart policy does not help there: the configuration is still wrong, so the service reaches the same fault again, and you are left watching it flap instead of seeing it plainly down with the reason waiting in the log.

There is a second reason, and it decides whether a restart policy does anything at all. Windows runs recovery actions when a service dies without reporting that it stopped. A service that does report stopped, and carries an error code with it, counts as a failure only once you turn the failure flag on as well. Which of those two a fault here produces is worth checking on your own box before you rely on a restart policy, because the difference decides whether it ever fires.

If you want the restart policy anyway, set it yourself, once, after installing:

```
sc.exe failure DucksInARow reset= 86400 `
actions= restart/5000/restart/10000/restart/30000
```

That restarts the service after five seconds on the first failure, ten on the second and thirty thereafter, and forgets the failure count after a day. Add the failure flag if you also want a stop that reports an error to count:

```
sc.exe failureflag DucksInARow 1
```

Neither is a substitute for reading the log. A service that keeps failing on a bad configuration keeps failing after a restart, and the reason is in `logs\ducks-<date>.log` in the data folder.

After installing

The service serves the setup wizard until you complete it, and every CA operation answers 503 until then. Open `https://your-server:5001` and work through the wizard; the [quickstart](#) walks through each step.

Then confirm the install is healthy with [verifying your installation](#).

4. Configuration

Every setting the service reads, where to put it, and which file wins.

Where settings live

Configuration is layered. Later sources override earlier ones:

1. `appsettings.json` in the **installation** folder. Shipped defaults. Upgrades replace this file, so do not edit it.
2. `appsettings.{Environment}.json`, if one is present.
3. `settings.json` in the **data** folder. This is the one to edit. The setup wizard writes here, and it survives upgrades.
4. Environment variables.
5. Command line arguments.

Restart the service after editing a file. The overlay is not watched for changes, and the restart is the apply step.

Edit settings.json as an administrator. The data folder is writable only by SYSTEM and Administrators, so open the file from an elevated session. The service ignores the file, with a Critical line in the log, if anyone other than SYSTEM or Administrators owns it or may write it. See Troubleshooting.

Not everything lives in these files. The allowed domain list, the EAB enforcement mode, the revocation scope and the device attestation settings are stored separately and apply immediately, with no restart. Those are called out where they appear below.

A minimal `settings.json` after setup looks like this:

```
{
  "Certus": {
    "CaConnectionString": "CA-SERVER\\MyCA",
    "ExternalUrl": "https://ducks.yourdomain.local:5001"
  }
}
```

A setting that looks like it has a default may be null. Several keys ship in `appsettings.json` as an explicit JSON `null`, and the configuration binder applies that `null` over the value the code would otherwise start from. So the shipped file, not the code, decides the starting value for those keys. They are marked below.

Core settings

Setting	Default	Purpose
<code>Certus:CaConnectionString</code>	<code>null</code> , ships <code>null</code>	The CA, in <code>CAHOST\CA</code> Name form. The wizard sets it

Setting	Default	Purpose
<code>Certus:DatabasePath</code>	<code>ducks.db</code> in the data folder, ships null	The SQLite database file. A blank value opens a private temporary database per connection, which is why it is normalised after binding rather than trusted from the file
<code>Certus:ExternalUrl</code>	null, ships null	The base URL you expect clients to use. Recorded and checked at startup. When it is set, every URL the ACME server hands out takes its scheme, host and port from it, whatever host the client connected to; when it is not, ACME URLs follow the host the client connects to
<code>Certus:SyncIntervalMinutes</code>	5	How often the dashboard syncs certificates from the CA. Values below 1 are raised to 1
<code>Certus:RequestHistoryDays</code>	30	How far back the sync reaches for pending, denied and failed requests, so their detail pages can show the CA's own explanation. Issued and revoked certificates are always synced in full. 0 skips those three passes and empties the matching filters
<code>Certus:EnableWalMode</code>	true	SQLite write ahead logging. Set false on a filesystem that cannot support WAL, or when a backup tool needs a single file with no -wal and -shm sidecars. Applied in both directions on every start, so changing it converts the existing database
<code>Certus:UseMockCa</code>	false	Development only. The service refuses to start if this is true while a CA connection string is set
<code>Certus:SettingsOverlayPath</code>	<code>settings.json</code> in the data folder	Where the overlay above is read from

The server's own HTTPS certificate

Ducks in a Row can enrol and renew its own TLS certificate from the CA it is connected to. See [quickstart](#) for turning it on.

Setting	Default	Purpose
<code>Certus:HttpsCertificateThumbprint</code>	null	The certificate currently served. The wizard and the Settings page write it

Setting	Default	Purpose
<code>Certus:HttpsCertificateAutoRenewalEnabled</code>	<code>true</code>	Renew before expiry without being asked
<code>Certus:HttpsCertificateRenewalWindowDays</code>	<code>30</code>	How long before expiry to renew
<code>Certus:HttpsCertificateRenewalCheckIntervalHours</code>	<code>24</code>	How often to check

The last three are deliberately absent from `appsettings.json`, so they take the values above unless you add them.

ACME

Setting	Default	Purpose
<code>Certus:Acme:ExposeAllTemplates</code>	<code>false</code>	Break glass. Exposes every template the CA publishes, ignoring the wizard's selection. Leave it false; the open posture is logged as a warning at every start

Which templates are exposed is otherwise decided by the wizard and stored in `ducks-setup.json`. With no recorded selection nothing is exposed at all, which is deliberate: it fails closed.

`Certus:Acme:ExposeAllTemplates` does not widen what the dashboard may revoke. That is a separate setting; see [revocation](#).

Challenge validation egress

The server connects out to the name being validated for HTTP-01 and TLS-ALPN-01. `device-attest-01` never connects out at all.

Setting	Default	Purpose
<code>...ChallengeValidation:BlockPrivateRanges</code>	<code>false</code>	The RFC 1918 ranges are allowed, because an internal CA usually issues for exactly those addresses. Set true if everything you validate is public
<code>...ChallengeValidation:BlockLoopback</code>	<code>true</code>	Always refuse loopback
<code>...ChallengeValidation:BlockLinkLocal</code>	<code>true</code>	Always refuse link local, including the cloud metadata address
<code>...ChallengeValidation:BlockUniqueLocalIpv6</code>	<code>true</code>	Always refuse IPv6 unique local
<code>...ChallengeValidation:AdditionalBlockedCidrs</code>	<code>[]</code>	Extra ranges to refuse

Setting	Default	Purpose
<code>...ChallengeValidation:AllowRedirects</code>	false	Follow redirects during HTTP-01
<code>...ChallengeValidation:MaxValidationAttempts</code>	5	Attempts before a challenge fails
<code>...ChallengeValidation:PollIntervalSeconds</code>	5	How often the validation worker sweeps

The prefix is `Certus:Acme:ChallengeValidation:.` The [hardening guide](#) explains the tradeoffs.

Orders the CA holds

Setting	Default	Purpose
<code>Certus:Acme:PendingIssuance:PollIntervalSeconds</code>	60	How often to ask the CA about orders it is holding for manager approval. Clamped between 5 seconds and 1 hour

A template set to hold every request for approval is an ordinary ADCS configuration, not an error. The order stays processing until an operator approves or denies it at the CA, and this sweep is what finishes it.

Rate limiting

Setting	Default	Purpose
<code>Certus:RateLimiting:Enabled</code>	true	
<code>Certus:RateLimiting:NewAccountLimit</code>	10	Per window, acme-new-account
<code>Certus:RateLimiting:NewOrderLimit</code>	30	Per window, acme-new-order
<code>Certus:RateLimiting:PollLimit</code>	300	Per window, acme-poll: authorization and order polling
<code>Certus:RateLimiting:GeneralLimit</code>	100	Per window, acme-general: everything else
<code>Certus:RateLimiting:WindowSeconds</code>	60	
<code>Certus:RateLimiting:SegmentsPerWindow</code>	6	How many steps the window releases permits in
<code>Certus:RateLimiting:HelpUrl</code>	none	Documentation URL for a Link: rel="help" on a refusal

A non positive value while rate limiting is enabled is logged as a warning at startup.

Which rate limit refused a request

A refusal names the policy that produced it, so you can tell which of the four settings above to change. The name appears in three places:

- the detail of the ACME problem document the client receives, for example `Too many new account requests.`, which is what shows up in the client's output
- a `Retry-After` header, in whole seconds
- a Warning in the service log naming the policy, the path, the caller's address, and the setting to raise. That line is written at most once per policy per minute, and reports how many further refusals it did not log, so that a flood cannot fill the log file

Set `HelpUrl` to your own runbook to add an RFC 8555 section 6.6 `Link` header pointing at it, with the policy name as the fragment. Nothing is sent anywhere by default.

The ACME polling budget

`PollLimit` covers the two POST-as-GET endpoints a client repeats while it waits, `authz` and `order`. Their request count is set by how long validation takes rather than by how many certificates were asked for.

The default is 5 requests a second. A client polling once a second contributes 60 a minute for as long as it waits, so the default carries about five concurrent issuances polling that hard, or ten polling every two seconds. The two endpoints share the budget without doubling the cost, because RFC 8555 puts them in sequence: a client polls the authorization until the challenge settles, then the order after finalize. Raise `PollLimit` before raising `GeneralLimit` if clients are refused part way through an issuance; the refusal names which of the two it was.

Rate limits partition per source address

Every policy counts per caller address, so anything that makes many clients share one apparent address makes them share one budget. A reverse proxy, a NAT gateway, or a Kubernetes cluster egressing through one address will do this.

Behind a proxy, set `Auth:TrustedProxies` to the proxy's address. Until you do, forwarded headers are ignored entirely and every client counts as the proxy, so one fleet shares a single limit. The service says so at startup when `TrustedProxies` is empty.

Permits are released in `SegmentsPerWindow` steps across the window rather than all at once, so a refused fleet recovers gradually instead of retrying in lockstep. `Retry-After` reports one step, `WindowSeconds` / `SegmentsPerWindow`. A caller that spends its whole budget in a single instant still waits a full window, and will be told to retry sooner than that; retrying early costs one more refusal carrying a fresh `Retry-After`, which is the cheaper error than holding back a caller that could already proceed.

Refusals are immediate. Requests are never queued, because queueing on a window this long would hold a connection open until the window replenished instead of answering, which is worse for the client and for the server than an immediate refusal the client can act on.

Alerts

See [alerts](#) for what these do. Note the shipped `appsettings.json` writes this section with its full name as a single key, "Certus:Alerts", rather than nesting Alerts inside Certus. Both forms bind.

Setting	Default	Purpose
Certus:Alerts:Enabled	true	
Certus:Alerts:CheckIntervalMinutes	60	
Certus:Alerts:ThresholdDays	[30, 14, 7, 1]	Days before expiry to alert
Certus:Alerts:Smtp	null, ships null	Null means email alerts are off
Certus:Alerts:Webhook	null, ships null	Null means webhook alerts are off
Certus:Alerts:Crl:Enabled	true	Watch the revocation lists of the CAs in the chain
Certus:Alerts:Crl:CheckIntervalMinutes	60	
Certus:Alerts:Crl:FetchTimeoutSeconds	15	How long to wait for a distribution point
Certus:Alerts:Crl:MaxCrlBytes	33554432	The largest CRL to accept, 32 MB

Most of the SMTP block is editable from the Settings page. The webhook block is deliberately not, so a dashboard administrator cannot redirect alert traffic.

Authentication

The dashboard and the setup API use Windows Integrated Authentication and are limited to an administrator group.

Setting	Default	Purpose
Auth:Mode	Negotiate	The only other legal value is Disabled, which the service refuses to start with while a CA is configured
Auth:AdminGroup	null, ships null	The group allowed in. Null means the built in Administrators group
Auth:RequireHttps	true	HSTS and an HTTP to HTTPS redirect outside development
Auth:TrustedProxies	[]	Reverse proxy addresses whose X-Forwarded-* headers are trusted

Web server and logging

Setting	Default	Purpose
Kestrel:Endpoints:Http:Url	http://0.0.0.0:5000	
Kestrel:Endpoints:Https:Url	https://0.0.0.0:5001	
Kestrel:Endpoints:Https:Certificate:Path	empty	Empty means the self signed certificate in the data folder is served
Serilog:MinimumLevel:Default	Information	Logs go to logs\ducks-<date>.log in the data folder, 30 daily files retained

What the service refuses to start with

Two combinations are refused outright, because each one silently disables a security control that the presence of a CA says you wanted:

- Auth:Mode=Disabled while Certus:CaConnectionString is set.
- Certus:UseMockCa=true while Certus:CaConnectionString is set.

Everything else that looks wrong is logged as a startup warning and the service runs. Read logs\ducks-<date>.log after a configuration change; the warnings name the setting.

5. ADCS Configuration

What to set up on the certificate authority side. Most of this is done once, and the setup wizard handles the parts it can.

Finding your CA connection string

The connection string is `hostname\CA Name`. The wizard finds it for you, so you rarely need to build one by hand. These are the fallbacks.

Let the wizard discover it. Open `https://your-server:5001` after installing. The wizard lists the CAs published in Active Directory and lets you pick one. It then tests the connection through the same DCOM path the service uses in production, as the service's own account, and checks what else that account may do on the CA (see [the service's rights](#)). This is the recommended route and needs nothing typed.

Read it off the CA. Open the Certification Authority console (`certsrv.msc`) on the CA server. The CA name is the node in the console tree. The connection string is the CA server's hostname, a backslash, then that name.

Ask the forest. On any domain joined machine:

```
certutil -config - -ping
```

That lists the available CAs with their connection strings already in the right form.

Use the fully qualified host name. The short form has not been verified to work.

A CA name may contain a comma or a space. Both are legal and both are handled. Quote the whole string when you pass it to a command line tool.

Certificate template permissions

The service runs as LocalSystem, so the identity the CA sees is the server's machine account, written `DOMAIN\SERVERNAME$`. That account needs **Enroll** on every template you want to expose over ACME.

Template permissions live in the Certificate Templates console, not in the Certification Authority console:

1. On the CA server, run `certtmpl.msc`. You can also get there from `certsrv.msc` by right clicking **Certificate Templates** and choosing **Manage**.
2. Right click the template and choose **Properties**.
3. Open the **Security** tab.
4. Add the Ducks in a Row server's machine account, for example `DUCKS-SERVER$`. Machine accounts do not appear in the object picker until you add **Computers** to the object types.
5. Grant **Enroll**. Do not grant **Autoenroll**: nothing here autoenrols, and granting it widens what the account can do for no benefit.
6. Click OK, and repeat for each template you intend to expose.

The wizard's template step checks each published template for ACME readiness and tells you what is missing. It does not check this permission. The Review step does: it reads each chosen template's permissions as the service's account and reports Enroll as **Inferred**, which is a reading and not a proof. Only an enrolment proves it. In the wizard that is the HTTPS certificate on the External URL step, for the template it uses; otherwise it is the first ACME order.

CA read permission

The machine account also needs **Read** on the CA itself: Certification Authority console, right click the CA, **Properties**, **Security** tab.

Read is what lets the dashboard sync the certificate inventory. Without it enrolment still works and clients get certificates, but the dashboard stays empty, which reads like a broken install and is not one.

Issue and Manage Certificates is a separate right, needed only for revocation from the dashboard. See [revocation](#) before granting it.

Template URL mapping

Each template is its own ACME endpoint:

```
https://your-server:5001/acme/<template>/directory
```

<template> is the template's programmatic name, which is its Active Directory cn, or its display name. Display names containing spaces work; the client URL encodes them. A display name carrying an invisible character, such as a soft hyphen pasted in from a word processor, is refused with a 400 rather than silently mismatching. Use the programmatic name in that case, and see [troubleshooting](#).

No template is reachable over ACME until the wizard records a selection. On a fresh install every template's directory URL answers 403 until then, which is deliberate.

Key algorithm

An ACME client must request the key type the template asks for, and templates differ. A template built on an RSA provider needs an RSA key; an ECDSA_P256 template needs a key on that curve. Requesting the wrong type is refused by the CA policy module at finalize. See [reading a denial](#) below for what that refusal tells you.

You do not have to work this out yourself. The wizard's template step reports the key algorithm it read from the template, and the **Client setup** snippets on the ACME page of the dashboard are generated with the right flags for that template already filled in. Copy them from there rather than from memory.

See [connecting ACME clients](#) for the per client flags.

Reading a denial

When ADCS refuses a request it records two things: its own message, and a status code giving the reason. Ducks in a Row reports both, because the message on its own is often worth very little. A template Enroll

denial on a stock Windows CA answers with the bare string `Denied by Policy Module` and nothing else.

The reason appears wherever the refusal does: in the setup wizard, in the service log at

`C:\ProgramData\Ducks in a Row\logs\ducks-<date>.log`, and in the problem document an ACME client receives. It reads like this:

Denied by Policy Module. 0x80094012: The permissions on the certificate template do not allow the current user to enroll for this type of certificate.

The codes you are most likely to meet:

Code	What to do
0x80094011	The CA's own permissions refuse the account. Grant Request Certificates on the Security tab of the CA in <code>certsrv.msc</code> .
0x80094012	The template's permissions refuse the account. Grant Enroll as described under certificate template permissions .
0x80094014	A certificate manager denied a request that was waiting for approval. Nothing is misconfigured; this was a decision.
0x80094800	The template is not published by this CA. Add it under Certificate Templates in <code>certsrv.msc</code> , and check you named the template rather than its display name.
0x80094811	The key is smaller than the template's minimum. The wizard's template step reports that minimum.
0x8009480A	The template requires enrollment agent signatures, which an ACME request never carries. Use a different template.

A code Ducks in a Row does not have a description for is still reported as a number. `certutil -error 0x80094012` names any of them.

To confirm the same values at the CA itself:

```
certutil -view -restrict "RequestId=109" -out "Request.DispositionMessage,Request.StatusCode"
```

Templates that hold requests for approval

A template configured to require manager approval answers every submission "pending" rather than issuing.

That is an ordinary ADCS setting, and it works: the ACME order stays in processing until an operator approves or denies the request at the CA, and Ducks in a Row finishes the order when that happens.

The wizard warns when a template you select is configured this way, so the behaviour is not a surprise. Nothing needs configuring for it to work.

6. Verifying Your Installation

Seven checks, in the order worth running them. Each one says where to go next when it fails, which for most of them is a section of [troubleshooting](#).

The installed files carry our signature

Every .exe and .dll the installer puts in the application folder is signed by Haruspex Systems B.V., so two commands tell you whether the binaries on disk are the ones we published. Point them at the folder you installed into if you changed it.

```
$installed = Get-ChildItem "C:\Program Files\Ducks in a Row" `
    -Recurse -Force -File -Include *.exe, *.dll
$installed.Count # how many were checked. Zero means the path is wrong

$installed | Get-AuthenticodeSignature | Where-Object {
    $null -eq $_.SignerCertificate -or
    $_.Status -ne 'Valid' -or
    $_.SignerCertificate.Subject -notlike 'CN=Haruspex Systems B.V.,*'
} | Format-List Path, Status, StatusMessage
```

A count with no rows under it is the answer you want. -Force is part of the check and not a detail: without it a file with the hidden attribute is skipped in silence. Keep the null test first as well: the two tests after it read a certificate that an unsigned file does not have.

A row is not proof of tampering on its own, so read it before acting.

- **HashMismatch, or Valid for a file signed by somebody else.** The file is not the one we published. Run Get-AuthenticodeSignature on it by itself to see who signed it, then stop the service, keep the file and logs\ducks-<date>.log as they are, and write to security@haruspex.systems about that first occurrence rather than reinstalling over the evidence.
- **The same complaint on every file at once,** usually UnknownError or an untrusted chain. That is normally something duller: a server with no internet access, which cannot finish building or checking our certificate's chain. Compare the installer against the SHA256 checksum instead, as [checking the download](#) describes. One file alone saying it belongs in the case above instead: a chain this server cannot check would not leave the rest of them clean.

This covers the binaries and not the rest: configuration files and the dashboard's own files are installed unsigned. Every binary it does cover carries the same publisher, which is what lets one publisher rule in an App Control for Business (WDAC) or AppLocker policy allow all of them, and this check is how you confirm that before writing the rule.

The service is running

```
Get-Service DucksInARow

# Status Name DisplayName
```

```
# -----  
# Running DucksInARow Ducks in a Row Certificate Proxy
```

A service that is not running has almost always logged the reason. Read `logs\ducks-<date>.log` in the data folder before anything else.

Health

```
Invoke-RestMethod https://your-server:5001/health
```

State	Meaning
Healthy	The database is reachable and so is the CA
Degraded	The database is fine, the CA is not reachable
Unhealthy	The database is not reachable

Degraded is the interesting one: the service is up and answering, but no certificate can be issued until the CA comes back. Alerts and the dashboard keep working, and ACME clients get a 503 that tells them to retry rather than an error that makes them give up.

There is a second endpoint, `/health/live`, which reports only that the process is up and answering. Use it for a load balancer probe, where a CA outage should not take the server out of rotation. Both endpoints are anonymous, so a monitor does not need credentials.

The ACME directory answers

```
Invoke-RestMethod https://your-server:5001/acme/WebServer/directory | ConvertTo-Json
```

Substitute a template you actually enabled. A 403 here means the template is not enabled for ACME, and a 404 means the name does not match a published template. Both are covered in [troubleshooting](#).

The firewall rules exist

```
Get-NetFirewallRule -DisplayName "Ducks in a Row*" |  
Select-Object DisplayName, Enabled, Direction, Action
```

The installer opens inbound TCP 5000 and 5001. If you changed the ports in `settings.json`, the rules still name the old ones and you need to add your own.

The dashboard loads

Open `https://your-server:5001` in a browser, signed in as a member of the administrator group.

The HTTPS endpoint uses a self signed certificate out of the box, so the browser warns on the first visit. That is expected. See [quickstart](#) for giving the server a certificate your clients already trust.

If the dashboard loads but the certificate inventory is empty, the machine account is probably missing **Read** on the CA. Enrolment works without it and the inventory does not: Request Certificates, which Authenticated Users hold on a default CA, does not open the certificate database. So an empty dashboard next to working issuance points straight at that permission. See [ADCS configuration](#).

The CA connection works

The wizard's connection step has a **Test Connection** button. It reads two properties from the CA through the same DCOM path the service uses in production, as the service's own computer account. The CA answers those reads only to an account holding **Request Certificates**, so a pass proves the network path, DCOM, and that one right.

It proves nothing about the other rights. On a default CA every domain computer holds Request Certificates through Authenticated Users, so a pass is the start of the answer, not all of it. The rights check below covers the rest.

If setup is already complete the wizard will not reopen. Use the health endpoint above, where Degraded is the same signal, or the rights check on the Settings page.

The service's rights

The wizard checks what the service's own account may do. The CA's rows appear as soon as Test Connection passes, and the Review step checks everything again, with one row for each template you chose. The **Service rights on the CA** card on the Settings page runs the same check against the configured CA and the templates enabled for ACME. **Check again** reruns it, for when a right has been granted or taken away since.

 **Service rights on the CA** Check again

What this server's account may do on the CA and on each template enabled for ACME, as the service itself found it. Proven means the right was used and worked; Inferred means it was read from a permission list or reported by the CA, and has not been used yet.

Checked as CORP\DUCKS015, this server's computer account, which is what the CA sees.

 Simulated: the demo CA answers for an example estate, so these results say nothing about a real CA.

-  **Domain membership** **PROVEN**
 This server is joined to the CORP domain.
 Needed for issuance, the certificate inventory, revocation and CRL watching.
-  **ADCS components on this server** **PROVEN**
 CertRequest, CertView and CertAdmin all start on this server.
 Needed for issuance, the certificate inventory, revocation and CRL watching.
-  **Reaching the CA as this server** **PROVEN**
 The CA answered Test Connection's property reads over DCOM, as this server's account.
 Needed for issuance and CRL watching.
-  **Request Certificates on the CA** **PROVEN**
 Test Connection's reads need this right, and the CA answered them.
 Needed for issuance and CRL watching.
-  **The certificate view the inventory reads** **PROVEN**
 The view the inventory sync reads opened for the service's account.
 Needed for the certificate inventory.
-  **Issue and Manage Certificates on the CA** **NOT GRANTED (OPTIONAL)**
 The CA reports that the service's account does not hold this right, so revoking from the dashboard and through ACME revoke-cert will fail. Everything else works without it.
 What to ask for: Grant CORP\DUCKS015 Issue and Manage Certificates on the CA (the Certification Authority console, CA Properties, Security) only if Ducks in a Row should revoke. It also lets the account approve and deny requests, so it is a real decision.
 Needed for revocation.
-  **Enroll on the Web Server template** **INFERRED**
 The template's permissions grant Enroll to Authenticated Users. The CA's policy module makes the final decision.
 Needed for issuance.
-  **A certificate issued to this server** **SKIPPED**
 The demo CA issues without checking any right, so nothing it issues proves one.
 Needed for issuance.
-  **Reaching the hosts being validated** **UNPROVEN**
 http-01 validation reaches each host on TCP 80, tis-alpn-01 on TCP 443, and dns-01 needs a DNS server the service can query on port 53. Which of them matter depends on your clients, so nothing here can check them ahead of time. device-attest-01 needs none.
 Needed for issuance.

Checked 9/26/2026, 9:35:26 AM

The Service rights on the CA card on the Settings page, showing a simulated report from the demo CA

Each row reads one of five:

Status	Meaning
Proven	The right was used and it worked, either just now or by the HTTPS certificate this CA issued to the server
Inferred	Read, not used: reported by the CA, or read from a template's permissions. Never a pass
Unproven	Nothing here could check it, or the check did not answer in time
Failed	Used and refused, or read and definitely absent. The row says what to ask for
Skipped	Not attempted, because something it depends on failed first

How each right is established:

- **Request Certificates on the CA** is proven by Test Connection itself.
- **The certificate view the inventory reads** is proven by opening it. The CA opens it to an account holding **Read**, and Request Certificates alone does not.

- **Issue and Manage Certificates** is reported by the CA, which reports the roles of any account holding Read or more. Only revocation needs this right, so its row is marked optional.
- **Enroll on a template** is read from the template's permissions, in the order Windows evaluates them. It stays Inferred until something enrolls from that template, because the CA's policy module decides with a token that can carry groups this server cannot see.

In the wizard, only the HTTPS certificate on the External URL step proves Enroll. It proves Request Certificates and Enroll on the template it uses. The wizard offers it only when the URL answers over HTTPS with a certificate the server does not trust. Otherwise the Settings page can provision one later, or the first ACME order is the proof. If you complete setup without it, the Review step asks you to acknowledge the rows it could not prove. It never blocks completion, because you may still be collecting the approvals.

The check runs inside the service, as LocalSystem, which the CA and the directory see as DOMAIN\SERVERNAME\$. It never checks the rights of the administrator at the browser, which would be the wrong answer. It only reads: nothing is requested, issued or revoked. On the dev host the report says **Simulated** and describes an example estate.

Issue a certificate

The real test is an end to end issuance. The [quickstart](#) has a ready to run certbot command, and the wizard's final step shows one already filled in for your server and template.

7. Connecting ACME Clients

Ducks in a Row implements RFC 8555, so any standard ACME client works. No agent or proprietary software is needed on your endpoints.

Running Kubernetes or OpenShift? cert-manager has a page of its own: [Kubernetes and OpenShift](#).

The directory URL

Every certificate template is its own ACME endpoint:

```
https://your-server:5001/acme/<template>/directory
```

- <template> is the template's programmatic name (its AD cn) or its display name. Display names with spaces are accepted; the client URL encodes them. A display name carrying an invisible character, such as a soft hyphen left behind by a paste from a word processor, is refused with a 400 malformed problem document; use the programmatic name and see [Troubleshooting](#).
- Use the HTTPS endpoint (5001) in production. Plain HTTP (5000) is for lab use only.
- Whether registration needs an external account binding credential depends on the server's enforcement mode; see [External account binding](#) below. Out of the box the mode is Off and clients register an account from their own key the first time they connect.

Trusting the server's TLS. ACME clients reject an untrusted TLS certificate on the ACME server itself. Give Ducks in a Row a certificate your clients already trust (for example, one chained to your internal CA root), or add that root to each client's trust store. The examples below note the per client switch for a lab where the server still uses its self signed certificate.

The example commands are representative. Exact flags vary by client version, so check each client's own documentation for your release.

External account binding

The ACME page of the dashboard sets one of three enforcement modes for external account binding (RFC 8555 section 7.3.4):

Mode	What a registering client needs
Off (default)	Nothing. A presented binding is ignored.
Optional	Nothing, but a presented binding is verified and recorded.
Required	A valid EAB credential; registration without one is refused with externalAccountRequired.

An administrator creates credentials on the ACME page. Each credential is a **key id** plus an **HMAC key** (base64url encoded, sized for HS256, which is what every client below signs with by default; HS384 and

HS512 are also accepted). The HMAC key is shown exactly once, right after create or regenerate, so hand both values to the client operator then. The binding happens once, at account registration; later orders and renewals ride the existing account while the credential stays active (revoking a credential, or letting it expire, suspends orders from the accounts bound to it). Accounts that registered before the mode was raised to Required keep working; the ACME page lists them as Unbound.

A credential can also carry a **domain namespace**: accounts bound to it may only order certificates inside those domains, across every template. Orders outside it are refused with `rejectedIdentifier` and a message naming the credential.

The dashboard writes ready to paste setup for you: the panel shown right after create or regenerate inlines the real secret, and each credential row's **Client setup** action shows the same snippets with a placeholder for the saved secret. The dashboard covers the first six clients below; Traefik is listed here for its field names only. The flags per client:

Client	Directory URL	Key id	HMAC key
certbot	--server	--eab-kid	--eab-hmac-key
win-acme	--baseuri	--eab-key-identifier	--eab-key
acme.sh	--server, once with --register-account	--eab-kid	--eab-hmac-key
Posh-ACME	Set-PA Server -DirectoryUrl	New-PA Account -ExtAcctKID	New-PA Account -ExtAcctHMACKey
cert-manager	spec.acme.server	externalAccountBinding. keyID	externalAccountBinding. keySecretRef, a secret holding the base64url key; see the EAB secret
Caddy	acme_ca	acme_eab block, key_id	acme_eab block, mac_key
Traefik	caServer	eab.kid	eab.hmacEncoded

If a client with correct looking values is refused with `unauthorized`, the credential may be revoked, expired, or rotated; see [troubleshooting](#) for the causes and fixes.

Challenge types

Type	Good for	How Ducks in a Row validates it	Outbound port it uses
HTTP-01	Standard web servers	Fetches <code>http://<domain>/.well-known/acme-challenge/<token></code>	80

Type	Good for	How Ducks in a Row validates it	Outbound port it uses
DNS-01	Wildcards, hosts with no inbound HTTP	Looks up the TXT record at <code>_acme-challenge.<domain></code>	53
TLS-ALPN-01	TLS only environments	Opens a TLS connection on port 443 and checks the ALPN certificate	443

The Ducks in a Row server is the party that performs validation, so the server needs outbound reachability to the domain or DNS being validated.

Key type

Template records	Ask for	Notes
An RSA provider	RSA, 2048 bits or more	The stock Web Server ACME template. Most clients default to an elliptic curve key, so this is the case that needs a flag
ECDSA_P256, ECDSA_P384, ECDSA_P521	An ECDSA key on the named curve	
ECDH_P256, ECDH_P384, ECDH_P521	An ECDSA key on the same curve	See below

An ECDH_* template is more common than it looks. The Certificate Templates console records ECDH when **Request Handling** has its purpose set to "Signature and encryption", which is a default rather than an unusual choice. A PKCS#10 certificate request cannot carry an encryption only key, so the answer is the signature key on the same curve, and the dashboard says as much when it generates the snippet.

Forcing RSA where the template is RSA:

Client	Flag
certbot	<code>--key-type rsa --rsa-key-size 2048</code>
lego	<code>--key-type rsa2048</code>
acme.sh	<code>--keylength 2048</code>
dehydrated	<code>KEY_ALG0="rsa"</code>
win-acme	<code>--csr rsa</code>

cert-manager asks for RSA 2048 unless told otherwise, so an RSA template needs nothing, and an ECDSA template needs `privateKey` on every Certificate. See [key type](#) on the Kubernetes page.

CSR requirements

At finalize the server checks the CSR against the order before anything is submitted to the CA:

- The subject alternative names must be DNS names matching the order's domains exactly, wildcard marker included. Any other SAN type (an IP address, an email, a UPN, a `directoryName`, a `PermanentIdentifier`) is refused.
- The subject common name is optional. When a CN is present, it must be one of the domains on the order; a descriptive value such as `CN=My Web Server` is refused. Public ACME CAs enforce the same rule, so a client that works against Let's Encrypt needs no change.
- A CSR with no SAN at all is identified by its subject CN, which then has to match the order the same way.

A CSR that breaks these rules is refused with `badCSR`, and the order stays in the ready state so the client can retry with a corrected CSR.

`badCSR` means the CSR itself, and only the CSR. A `finalize` on an order that is no longer ready is refused with `orderNotReady` instead, including the case where another request finalized the same order first. Rebuilding the key and the CSR cannot help there; re-read the order and act on the status it reports.

A CA that refuses the request is a different answer again. The `finalize` returns 500 with `serverInternal`, the detail carries the CA's own disposition message, and the order becomes invalid: the CA has decided, and no CSR will change its mind. Read the detail, fix the template or the key type, and start a new order.

That same problem document is also on the order itself, in the `error` member RFC 8555 section 7.1.3 defines, so a client that lost the `finalize` response or that polls rather than reads it can fetch the order and find the reason there. It reads word for word the same either way.

One case leaves the order's `error` empty on purpose: an order that went invalid because a challenge failed. The reason for that failure belongs to the authorization, so follow the order's `authorizations` URLs and read the error on the failed challenge, which is what section 7.1.6 asks a client to do.

If the CA cannot be reached at all, the `finalize` returns 503 with `serviceUnavailable` and the order is left alone. Retry the same `finalize` once the CA is back. Nothing is lost, and the authorizations you already completed still stand.

Renewal timing (ARI)

The directory advertises `renewalInfo`, the ACME Renewal Information extension (RFC 9773). A client that understands it asks the server when to renew each certificate instead of guessing from the expiry date, with a plain unauthenticated GET of `{renewalInfo}/{certID}`, where the identifier is built from the certificate's Authority Key Identifier and serial number exactly as the RFC describes. No account or signature is needed.

The response is a suggested window sitting at roughly two thirds of the certificate's lifetime, about 2% of that lifetime wide, with a per certificate offset derived from the serial number. The offset is what keeps a fleet issued in one batch from renewing in one batch, and it is deterministic, so the window a client sees never moves between polls. The `Retry-After` header says how often to ask again (six hours).

Two behaviors are worth knowing about:

- A **revoked** certificate answers with a window entirely in the past, which an ARI aware client reads as "renew immediately". This is how a revocation reaches your fleet without anyone touching the clients, and it works however the certificate was revoked: through ACME, from the dashboard, or directly at the CA with `certutil` or `certsrv.msc`.
- When renewing, a conforming client adds a `replaces` member to its new-order request naming the certificate it is replacing. The server verifies it (same account, at least one shared identifier) and reflects it on the order. A second live order naming the same certificate is refused with HTTP 409 and `urn:iETF:params:acme:error:alreadyReplaced`, which stops duplicate renewal loops; if that happens, the earlier order is the one to complete. An earlier order that was abandoned stops blocking once it passes its own expiry, so a crashed renewal attempt never locks a certificate out.

Clients that read ARI include `certbot` from 4.1.0, `lego` (checked by default since 4.20.2), `Caddy` from 2.8.0, and `simple-acme`, which follows it unless `ScheduledTask.RenewalDisableServerSchedule` is set. `cert-manager` reads it only from 1.21, and only with its alpha `ACMEUseARI` feature gate switched on; see [renewal and ARI](#). Clients that do not simply ignore the directory member and renew on their own schedule, as before.

certbot

```
# HTTP-01, certbot answers the challenge itself on port 80
certbot certonly --standalone \
  --server https://your-server:5001/acme/WebServer/directory \
  --email you@example.com \
  -d host.corp.example.com \
  --key-type rsa --rsa-key-size 2048
```

Match the key type to the template. Your client must ask for the key type the template wants, and templates differ. Ask for the wrong one and the CA policy module refuses at finalize with `Denied by Policy Module`, which reads like a permissions problem and is not one. The wizard's template step reports the key algorithm it read from the template, and the **Client setup** snippets on the dashboard's ACME page come with the right flags already filled in for that template. That is the reliable answer for your CA; the table under [key type](#) is the general shape.

For a lab server with an untrusted TLS certificate, set `REQUESTS_CA_BUNDLE` to your CA root PEM so `certbot` trusts the endpoint, or install that root into the system trust store.

win-acme (Windows)

Run `wacs.exe` and, in the menu, set the ACME server to the template directory URL, then create a certificate. Unattended example:

```
wacs.exe --source iis --siteid 1 ^  
--baseuri https://your-server:5001/acme/WebServer/directory
```

win-acme uses the Windows certificate trust store, so import your CA root on the machine running it if the server certificate is not already trusted.

Caddy

```
{
  acme_ca https://your-server:5001/acme/WebServer/directory
  # For an internal CA, point Caddy at the root so it trusts the ACME endpoint
  acme_ca_root /etc/ssl/certs/corp-root.pem
  email you@example.com
}

host.corp.example.com {
  respond "Hello from Caddy"
}
```

Traefik

```
certificatesResolvers:
  ducks:
    acme:
      caServer: https://your-server:5001/acme/WebServer/directory
      email: you@example.com
      storage: /etc/traefik/acme.json
      httpChallenge:
        entryPoint: web
```

Traefik uses the host or container trust store. Add your CA root there if the ACME endpoint certificate is not already trusted.

Posh-ACME (PowerShell)

```
Import-Module Posh-ACME

# Point Posh-ACME at the template directory.
# Add -SkipCertificateCheck only for a lab server with untrusted TLS.
Set-PAServer -DirectoryUrl https://your-server:5001/acme/WebServer/directory

New-PAAccount -Contact you@example.com -AcceptTOS

New-PACertificate -Domain host.corp.example.com -Plugin WebRoot `
  -PluginArgs @{ WebRootPath = 'C:\inetpub\wwwroot' }
```

cert-manager (Kubernetes)

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: ducks-in-a-row
spec:
  acme:
    server: https://your-server:5001/acme/WebServer/directory
    caBundle: <base64 of your CA root, PEM>
    email: you@example.com
    privateKeySecretRef:
      name: ducks-account-key
    solvers:
      - http01:
          ingress:
            ingressClassName: nginx
```

cert-manager trusts only public roots unless `caBundle` names yours, and it asks for an RSA 2048 key unless a Certificate says otherwise. Issuer or ClusterIssuer, the EAB secret, the path an HTTP-01 check takes through your ingress, and what is known about OpenShift are on the [Kubernetes and OpenShift](#) page.

Picking a template

Any template that was enabled in the setup wizard is reachable at `/acme/<template>/directory`. Match the template to the certificate purpose, for example a Web Server template for TLS server certificates. A template the CA publishes but that was not enabled in the wizard answers 403 not enabled for ACME; if a directory URL returns 404 unknown certificate template, check the name against the templates shown in the setup wizard.

8. Kubernetes and OpenShift

Workloads in a Kubernetes cluster get their certificates from cert-manager, and often from a CA the cluster keeps for itself, which nothing outside the cluster trusts. cert-manager's built in ACME issuer can ask Ducks in a Row instead. The certificates then chain to the ADCS root your organisation already trusts. The cluster needs no plugin, holds no Active Directory account and no rights on the CA, and one scoped credential can decide which names it may order.

This page covers cert-manager on Kubernetes, and what is known about OpenShift. For other clients, see [connecting ACME clients](#).

What has been run. Our regular test cycle runs cert-manager against a real ADCS certificate authority: a namespaced Issuer, a Certificate with an RSA key, the Ingress annotations, HTTP-01 through ingress-nginx, and DNS-01. Not yet run against Ducks in a Row: trusting the server over HTTPS with caBundle, external account binding, a ClusterIssuer, an ECDSA template, renewal and ARI, a challenge that fails, and anything on OpenShift. Sections that rely on those say so, and what they describe is read from the server's code and from cert-manager's own code and documentation.

Before you start

On the Ducks in a Row side:

- The template is enabled for ACME in the setup wizard. One that is not answers 403, "not enabled for ACME".
- If the allowed domain list is on, the cluster's host names are on it.
- The external account binding mode suits you, and a credential exists if the mode is Required or you want the cluster's orders limited to its own domains. See [external account binding](#).
- The server's External URL is the name the cluster will use to reach it. See [pointing at a template](#).
- The server has enrolled its own HTTPS certificate from your CA. See [giving clients a TLS certificate they trust](#).
- The server can reach the cluster's ingress on TCP 80, and its DNS resolves the certificate names to that ingress. See [HTTP-01 through your ingress](#).

On the cluster side:

- cert-manager 1.11 or later, the first release with caBundle. Use a release cert-manager still supports.
- An ingress controller that answers plain HTTP on port 80.

Issuer or ClusterIssuer

Each one is an ACME account on the server. They differ in who can use them and where their secrets live.

	Issuer	ClusterIssuer
Issues for	Certificates in its own namespace	Certificates in every namespace

	Issuer	ClusterIssuer
Reads its secrets from	Its own namespace	The cluster resource namespace: <code>cert-manager</code> , unless the controller's <code>--cluster-resource-namespace</code> flag changed it
Accounts on the server	One per Issuer	One for the cluster

Not yet run against Ducks in a Row: a ClusterIssuer. Nothing on the server tells the two apart; both register an account the same way.

Choose by who owns the host names. If each team owns its own names, give each team's namespace its own Issuer and each team its own credential, with a domain namespace listing that team's domains. A team can then order only its own names, and revoking one team's credential leaves the others alone. If one platform team owns every name, a single ClusterIssuer with one credential is simpler.

The word namespace means two unrelated things here. A Kubernetes namespace is where an Issuer and its secrets live. A domain namespace belongs to an EAB credential and lists the DNS domains its accounts may order; see [domain namespaces](#). The pattern above maps one of each to one team.

Start from the dashboard manifest

The dashboard writes the manifest for you. On the **ACME** page, open **Credentials**, create a credential, and pick **cert-manager** in the setup panel. The panel shown right after create inlines the real HMAC key. Later, a credential row's **Client setup** shows the same manifest with a placeholder where the key goes, because the key is shown only once.

The manifest holds three things:

- the `kubectl create secret` line that stores the HMAC key,
- an Issuer pointed at the template's directory, carrying the credential's key id and a `caBundle` placeholder,
- a sample Certificate whose `privateKey` already matches the template.

Before you apply it:

1. Replace the `caBundle` placeholder; see [trusting the ACME endpoint](#). Left in place, it makes `kubectl apply` refuse the manifest.
2. Set the email, and set `ingressClassName` to your ingress controller's class.
3. Pick the namespace, and give the secret line and `kubectl apply` the same `-n`.
4. For a ClusterIssuer, change the Issuer's kind and the Certificate's `issuerRef` kind to `ClusterIssuer`, and create the secret in the cluster resource namespace instead.

The Issuer it produces looks like this:

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
```

```
name: ducks-in-a-row
namespace: team-a
spec:
  acme:
    server: https://ducks.corp.example.com:5001/acme/WebServer/directory
    caBundle: <base64 of your CA root, PEM>
    email: you@example.com
    privateKeySecretRef:
      name: ducks-account-key
    externalAccountBinding:
      keyID: <key id from the dashboard>
      keySecretRef:
        name: ducks-eab
        key: secret
  solvers:
    - http01:
        ingress:
          ingressClassName: nginx
```

Leave out externalAccountBinding if the enforcement mode is Off and you are not using a credential.

The EAB secret

Not yet run against Ducks in a Row: external account binding with cert-manager.

Store the HMAC key exactly as the dashboard shows it:

```
kubectl -n team-a create secret generic ducks-eab --from-literal=secret='<HMAC key>'
```

- The key is already base64url encoded: 43 characters, no padding. Store it exactly as shown, and do not encode it again, even though cert-manager's own documentation shows an encoding step for keys that arrive raw. cert-manager decodes the stored value and uses the result as the HMAC key as it stands, so a key encoded twice, or stored with a trailing newline (as when a file written by echo is loaded with --from-file), gives the wrong key. The server then refuses the registration with 403 unauthorized and External account binding signature verification failed. A value that is not base64url at all fails earlier, inside cert-manager, with failed to decode external account binding key data. Use --from-literal, as above.
- The binding happens once, when cert-manager registers its account. From then on the account key, stored in the secret privateKeySecretRef names, is the cluster's identity. Keep it, and back it up with the cluster's other secrets.
- One credential can bind any number of accounts. A rebuilt cluster, a second cluster or an extra Issuer registers a new account against the same credential, and the ACME page lists every account bound to it.
- Regenerating a credential keeps its key id and every account already bound to it. Only a new registration needs the new key, but update the Kubernetes secret anyway so the next one works.

- An account stays bound to the credential it registered with. Pointing an existing Issuer at a different key id changes nothing on the server, and if the old credential is revoked, that account's orders are refused with `unauthorized`. To move an Issuer to a new credential, give `privateKeySecretRef` a new name as well, so `cert-manager` creates a new account key and registers it with the new binding.
- Revoking a credential, or letting it expire, suspends orders from every account bound to it; see [revoking, expiring, and the difference](#).

Pointing at a template

`spec.acme.server` is the template's directory URL:

```
https://ducks.corp.example.com:5001/acme/<template>/directory
```

- Use the host name in the server's External URL, exactly. The certificate the server enrolls for itself names that host and no other, so `cert-manager` refuses any other name for it. And once the External URL is set, every URL the server hands back, for new accounts, orders and certificates, is built from it, so the cluster has to resolve and reach that name whatever the directory URL says. The dashboard's manifest already uses it.
- Use HTTPS on port 5001. Plain HTTP on port 5000 is for lab use only, and by default it redirects to HTTPS.
- Every template is its own directory, so each template needs its own issuer. Name it by its programmatic name, its AD `cn`, to keep spaces out of the URL.

Trusting the ACME endpoint

Not yet run against Ducks in a Row: `caBundle`. Our test cycle reaches the server over plain HTTP.

Once the server has enrolled its own certificate, that certificate comes from your AD CS CA, and `cert-manager` trusts only the public roots in its container, so it has to be told about yours. Without that the Issuer never becomes ready, and reports `x509: certificate signed by unknown authority`. A server still serving its self signed certificate is no use here: that certificate names only `localhost` and the server's short machine name, not the External URL host, so enrol one first.

Put your root certificate in `caBundle`, as PEM, base64 encoded on one line:

```
base64 -w0 corp-root.pem  
[Convert]::ToBase64String([IO.File]::ReadAllBytes('corp-root.pem'))
```

To get the PEM, export the root on any domain joined Windows machine: in `certlm.msc`, open **Trusted Root Certification Authorities**, then **Certificates**, right click your root, choose **All Tasks**, **Export**, and pick **Base-64 encoded X.509**. With an issuing CA under an offline root, put the issuing CA's certificate in the bundle after the root, so the chain verifies even when the server does not send it.

- Never set `skipTLSVerify`. It turns off verification of the server entirely, and `cert-manager`'s own documentation marks it insecure.

- cert-manager is written in Go, which is stricter than Windows about two things older ADCS estates run into. It refuses a certificate signed with SHA-1 anywhere below the root, so a CA that still signs with SHA-1 has to move to SHA-256 first. And it never takes a host name from a certificate's common name, only from its subject alternative names. The certificate the server enrolls for itself carries the External URL host as a DNS name, so it passes; a certificate you install by hand has to as well.

HTTP-01 through your ingress

Ducks in a Row validates a challenge itself, from the Windows server it runs on. A challenge goes like this: cert-manager starts a small solver pod, routes `/.well-known/acme-challenge/<token>` to it through your ingress controller, checks that it can fetch the token itself, then tells the server it is ready. The server then fetches `http://<name>/.well-known/acme-challenge/<token>` on port 80.

What that asks of your network:

- The server must reach the ingress on TCP 80; see [network ports](#). In a segmented network this is the one firewall rule to ask for, from the server to the ingress addresses.
- The server resolves the name with its own DNS servers and connects to the first address it gets back. On the server, the name has to resolve to the ingress, not to an address only the cluster can reach.
- By default the server does not follow redirects (`AllowRedirects` is off). If your ingress sends plain HTTP to HTTPS for the whole host, exempt `/.well-known/acme-challenge/` from that, or the challenge fails with the redirect's status, for example server returned HTTP 308.
- By default a name that resolves to a loopback, link local or IPv6 unique local address is refused, and so is a name with any such address among its answers. If `BlockPrivateRanges` or `AdditionalBlockedCidrs` is set in [challenge validation egress](#), make sure the ingress addresses are still allowed.

cert-manager's own check, the one before it tells the server it is ready, runs from inside the cluster through the cluster's DNS. If the cluster cannot resolve your internal zone, the challenge sits pending and the server is never asked; our test cycle hit exactly this. Make the zone resolvable in the cluster, for example by forwarding it from CoreDNS to your DNS servers. From cert-manager 1.21 you can instead set `waitInsteadOfSelfCheck` on the solver, which skips the check and waits a fixed time.

HTTP-01 cannot prove a wildcard name, so a wildcard order is offered DNS-01 only. The server looks up `_acme-challenge.<name>` with its own DNS servers, so cert-manager's DNS-01 solver has to write into a zone those servers answer from. Setting one up for an internal zone is outside this page; cert-manager's documentation lists the providers, and our test cycle has run its RFC 2136 solver. The server also offers TLS-ALPN-01, but cert-manager has no solver for it.

Requesting a certificate

A Certificate asks for one:

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
```

```
name: app
namespace: team-a
spec:
  secretName: app-tls
  dnsNames:
    - app.corp.example.com
  issuerRef:
    name: ducks-in-a-row
    kind: Issuer
  privateKey:
    algorithm: RSA
    size: 2048
```

Or annotate an Ingress, and cert-manager creates the Certificate from the Ingress's `tls` section:

```
metadata:
  annotations:
    cert-manager.io/issuer: ducks-in-a-row
    cert-manager.io/private-key-algorithm: RSA
    cert-manager.io/private-key-size: "2048"
```

For a ClusterIssuer the annotation is `cert-manager.io/cluster-issuer`, and a Certificate names it with `kind: ClusterIssuer` in its `issuerRef`.

What the server accepts is set out under [CSR requirements](#). For a Certificate that means:

- DNS names only. The server refuses IP address identifiers and every other kind of subject alternative name, so leave `ipAddresses`, `uris` and `emailAddresses` unset.
- `commonName` is optional. If you set it, it has to be one of the `dnsNames`.
- `duration` changes nothing. The template sets the certificate's lifetime. cert-manager sends a requested lifetime only when the issuer sets `enableDurationFeature`, and the server then accepts the request and still issues for the template's lifetime.

Key type

The template decides the key type, and cert-manager defaults to RSA: with no `privateKey` it asks for RSA 2048. So an RSA template needs nothing, and an ECDSA template needs the setting on every Certificate, or the annotations on every Ingress.

Template records	privateKey
RSA, minimum 2048	Nothing, or <code>algorithm: RSA</code> and <code>size: 2048</code>
RSA, minimum above 2048	<code>algorithm: RSA</code> and <code>size: 4096</code> . cert-manager accepts only 2048, 4096 and 8192, so a 3072 minimum needs 4096
ECDSA_P256	<code>algorithm: ECDSA</code> and <code>size: 256</code>

Template records	privateKey
ECDSA_P384	algorithm: ECDSA and size: 384
ECDSA_P521	algorithm: ECDSA and size: 521
ECDH_P256, ECDH_P384, ECDH_P521	The ECDSA key on the same curve; see key type

Not yet run against Ducks in a Row: an ECDSA template. Our test cycle issues with an RSA key.

The dashboard's manifest fills this in for the template you pick, and the setup wizard shows the algorithm it read from the template.

A key the template does not accept is refused by the CA's policy module, not by Ducks in a Row. The finalize answers `serverInternal` carrying the CA's own message, typically `Denied by Policy Module` with the explanation that the public key does not meet the template's minimum size, and the order becomes invalid. cert-manager then waits before trying again, starting at an hour and doubling up to 32 hours. Fix the key settings, then run `cmctl renew <certificate>` to retry at once.

Trusting issued certificates in pods

The Secret cert-manager writes holds `tls.crt` and `tls.key`. The server returns the leaf, the issuing CA and the root, and cert-manager keeps what it downloads, so `tls.crt` carries that whole chain. The ACME issuer never writes `ca.crt`, so an application that insists on one has to be given the root another way.

Anything that connects to these workloads has to trust your ADCS root. Windows machines in the domain already do. Inside the cluster you distribute the root yourself:

- **A ConfigMap.** Put the root PEM in a ConfigMap in each namespace that needs it and mount it beside the certificate, for example with a projected volume that presents `tls.crt`, `tls.key` and the root in one directory. It needs no extra component, but keeping the copies current is up to you.
- **trust-manager.** cert-manager's companion project copies one bundle into every namespace, or into the namespaces a label selects. Its `Bundle` API is still `v1alpha1`, so follow its documentation for the release you install.

Renewal and ARI

cert-manager renews at two thirds of the certificate's actual lifetime unless you set `renewBefore` or `renewBeforePercentage`, and the lifetime is whatever the template gave the certificate. Since cert-manager 1.18, every renewal generates a new private key, because `rotationPolicy` defaults to `Always`.

The server advertises ACME Renewal Information (ARI), which lets it tell a client when to renew; see [renewal timing](#). Its suggested window sits at about the same two thirds point, so on an ordinary day ARI changes nothing for cert-manager. It matters after a revocation: a revoked certificate's window moves into the past, and a client that reads ARI renews at once.

cert-manager reads ARI only from release 1.21, and only with the ACMEUseARI feature gate switched on for the controller. The gate is alpha and off by default, so out of the box cert-manager renews on its own schedule, and a revocation does not reach the cluster until the next scheduled renewal. The gate goes in the controller's feature gates, for example this Helm value:

```
config:
  featureGates:
    ACMEUseARI: true
```

Not yet run against Ducks in a Row: renewal, and ARI with cert-manager.

The server limits requests per source address, and a cluster usually reaches it through one address, so every Issuer in it shares one budget. A burst of new certificates can be refused with `rateLimited`; see [rate limits partition per source address](#).

OpenShift

Nothing on this page has been run on OpenShift, and nothing about OpenShift itself has been verified against Ducks in a Row. The Issuer and Certificate above are standard cert-manager resources, and Red Hat ships cert-manager as the cert-manager Operator for Red Hat OpenShift; see Red Hat's [documentation for the operator](#). Two points are documented upstream and have not been run here:

- **HTTP-01 through the built in router.** Since cert-manager 1.18, the solver's temporary Ingress uses `pathType: Exact`. OpenShift converts an Ingress into a Route for its router and skips a path of that type, so the challenge URL is never served and the challenge fails with a 503. cert-manager's [compatibility notes](#) give the fix, switching off the `ACMEHTTP01IngressPathTypeExact` feature gate, and say the Red Hat operator has had that gate off by default since its 1.18.0 release. Documented upstream, not run here.
- **Routes.** cert-manager itself does not act on OpenShift Routes. The separate [openshift-routes](#) project issues certificates for Routes from any cert-manager issuer, and warns that it is not designed for clusters shared between tenants. Documented upstream, not run here.

Troubleshooting

Symptom	Likely cause	What to do
The Issuer never becomes ready: x509: certificate signed by unknown authority	cert-manager does not trust your root	Set <code>caBundle</code> ; see trusting the ACME endpoint
x509: certificate relies on legacy Common Name field, or a SHA-1 complaint	The server's certificate names its host only in the common name, or the chain is signed with SHA-1	Let the server enrol its own certificate, and move the CA to SHA-256
<code>externalAccountRequired</code>	The mode is Required and the Issuer has no binding	Add <code>externalAccountBinding</code> and the secret

Symptom	Likely cause	What to do
failed to decode external account binding key data	The secret is not base64url, for example standard base64 with +, / or = in it	Store the key exactly as the dashboard shows it
403 unauthorized: External account binding signature verification failed.	The key cert-manager holds is not the one issued: encoded a second time, stored with a trailing newline, or from before a regenerate	Recreate the secret from the key as shown, with --from-literal; see the EAB secret
403 unauthorized naming a credential	The credential is revoked, expired, or cannot be decrypted on this server	See troubleshooting
rejectedIdentifier	A name is outside the allowed domain list or the credential's domain namespace	Add the domain, or order within the namespace
403 "not enabled for ACME"	The template is not enabled for ACME	Enable it in the setup wizard, or point the Issuer at one that is
serverInternal with Denied by Policy Module	The key does not suit the template	Match privateKey to the template; see key type
A challenge stays pending and the server log shows no attempt	cert-manager's own check cannot fetch the token, often because the cluster cannot resolve the name	Make the zone resolvable in the cluster, or use waitInsteadOfSelfCheck
cert-manager keeps waiting after it accepted the challenge, then reports a timeout	The server's HTTP-01 attempt failed	Read the reason in the server log, fix it, then <code>cmctl renew</code> ; see below
rateLimited	Many certificates at once through one source address	Retry later, or see rate limits

A failed HTTP-01 check can look like waiting. This is read from the server's code and cert-manager's, and has not been run. When the server's HTTP-01 attempt fails, the server marks that challenge invalid, but it leaves the authorization pending while the other challenges it offered for the same name, DNS-01 and TLS-ALPN-01, are still unanswered. cert-manager waits on the authorization, not the challenge, for up to two minutes at a time, so it can report a timeout rather than the reason. The reason is in the server's log, in a warning of the form `Challenge <id> (http-01) for <name> failed: <reason>`, where the reason is something like a refused connection or `server returned HTTP 308`.

9. External Account Binding

External account binding (RFC 8555 section 7.3.4) decides **who may register an ACME account** with your server. This page is the administrator's half. For the client flags, see [connecting ACME clients](#).

Administration lives on the **ACME** tab of the dashboard.

The External Account Binding card on the ACME page, showing the three enforcement modes with Off selected

Enforcement modes

Mode	What a registering client needs
Off	Nothing. A presented binding is ignored. This is the default
Optional	Nothing, but a presented binding is verified and recorded
Required	A valid credential. Registration without one is refused with <code>externalAccountRequired</code>

The mode applies immediately when you change it. There is no restart, and an upgrade from a version without this feature reads as Off, so nothing changes underneath an existing install. Re-running the setup wizard on a configured server does not reset it either.

Raising the mode to Required does not lock out accounts that already exist. Accounts registered before the change keep working, and the ACME page lists them as Unbound so you can see which they are. If you want them gone, deactivate them deliberately; nothing does it for you.

Issuing a credential

A credential is a **key id** and an **HMAC key**. The key id is 32 hexadecimal characters. The HMAC key is sized for HS256, which is what every common client signs with by default, and HS384 and HS512 are accepted too.

The HMAC key is shown exactly twice in its life: when you create it, and when you regenerate it. It is not in any list, any error body, or any log line, and the API will not return it again. Hand both values to the client operator at the moment you create them.

The dashboard writes ready to paste client setup for you. The panel shown right after create or regenerate has the real secret inlined; each credential row's **Client setup** action shows the same snippets later with a placeholder where the secret goes.

Revoking, expiring, and the difference

Revocation is terminal. A revoked credential refuses edit, regenerate and owner change alike. There is no un-revoke. If you need the holder back, issue a new credential.

Expiry is not terminal. An expired credential is suspended, not dead. Move the expiry date forward, or clear it, and it works again.

Both suspend the accounts bound to them, and they do it at two points: when a new order is created, and at finalize. A suspended account gets 403 unauthorized.

The order itself stays in the ready state when this happens. That is deliberate: if you un-suspend the credential, the client can retry the same order rather than starting over.

Domain namespaces

A credential can carry a domain namespace, which limits what the accounts bound to it may order, across every template.

A namespace **narrows within the global allowed domain list, it never widens past it.** The global list is the ceiling and is checked first, so putting a domain in a namespace that the global list does not permit does not make it orderable. If you want a credential to reach a new domain, add it to the global list first.

A credential with no namespace is unrestricted, subject to the global list like any other.

An order outside the namespace is refused with `rejectedIdentifier` and a message naming the credential, and the refusal is recorded on the dashboard activity feed labelled as an EAB refusal, so it is distinguishable from a global allowed domain refusal.

Linking a credential to a person or a machine

A credential can record which Active Directory principal it was issued to. Users, computers, groups and service accounts can all be picked, including group managed service accounts, which are the usual identity for an automated ACME client.

This link is a record, not a control. Nothing checks it at issuance time. It exists so that six months later you can answer "who has this credential" without consulting a spreadsheet.

Moving the data folder breaks the secrets, on purpose

Credential secrets are encrypted with the machine's Data Protection keyring, which lives in `keys\` inside the data folder and is tied to the machine.

Restore the data folder onto a different machine and the secrets can no longer be decrypted. Verification then **fails closed**: clients get 403 unauthorized and the log says to regenerate.

This is the intended behaviour, not a defect. It means a copy of your data folder is not a copy of your credentials.

Recovery is straightforward, because everything except the secret survives the move. The key id, name, expiry, namespace and owner are all stored in the clear:

1. Open the ACME page on the new machine. Your credentials are all listed.
2. Regenerate each one. **The key id stays the same**, so only the HMAC key has to be redistributed.
3. Give each client operator the new HMAC key.

See [backup and restore](#) for how to avoid the situation.

EAB and device attestation do not mix

The Apple ACME payload has nowhere to put EAB credentials, so a device using `device-attest-01` cannot present one.

The consequence is concrete: **with enforcement set to Required, Apple devices cannot register at all**. That is not a bug and it is not fixable at this end.

Separate templates do not get round this. The enforcement mode is one setting for the whole server, not one per template, and an ACME account works on every template's directory, so Required refuses a new device on every template alike. An account a device registered before the mode was raised keeps working, but any new registration is refused, a device enrolling again included. If you use device attestation, keep the mode at Off or Optional; the device attestation allowlist, not EAB, is the gate for device orders. See [device attestation](#).

10. Device Attestation

Device attestation lets a hardware backed device prove its identity to the CA before it gets a certificate.

Ducks in a Row implements the ACME `device-attest-01` challenge from [draft-ietf-acme-device-attest-08](#), which extends RFC 8555 with two device identifier types and one attestation challenge.

In v1 the only supported attestation format is `apple`, produced by Apple Managed Device Attestation (MDA). An Apple device managed by your MDM proves, with a key that never leaves its Secure Enclave, that it is the specific device named in the order. Ducks in a Row verifies that proof against Apple's own attestation root before it asks ADCS to issue.

The feature is **off by default and fails closed**. Until an administrator creates a device attestation profile on a template, that template does not offer `device-attest-01` at all, and an order for a device identifier is refused as if the feature did not exist. Nothing you do elsewhere in the product turns this on by accident.

How it works

A device order carries a **permanent-identifier** rather than a dns name. Its value is the device serial number (or the UDID), compared octet for octet.

1. The MDM pushes an ACME certificate payload to the device, pointing it at one of your template directories.
2. The device orders a certificate for its own permanent-identifier. If the template has a profile and the identifier is allowed, the order gets a single `device-attest-01` challenge; otherwise the order is refused (see [Gate modes](#)).
3. The device builds an attestation object with its Secure Enclave key and posts it to the challenge. Ducks in a Row verifies it: the attestation chains to the Apple root, the freshness nonce matches this challenge, and the attested serial equals the order identifier.
4. At finalize the server binds the certificate request to the attestation: the CSR public key must be the exact key that was attested, and the CSR identity must match the order under the profile's [binding mode](#).
5. The request goes to ADCS on the profile's template, unchanged from any other ACME issuance.

The security model is a three way binding: a trusted attestation authority (Apple) vouches for the device, the attested key equals the key in the CSR, and the attested device identifier equals the order identifier. All three must hold.

Enabling it

Everything is configured on the **ACME** tab of the dashboard, in the **Device attestation** card. No wizard step and no configuration file entry is involved, so a setup wizard re-run never changes it.

1. Confirm the template you will use meets the [ADCS requirements](#) below and is published and ACME enabled.

2. Create a **profile** on that template. Pick a [gate mode](#) and a [CSR identifier binding](#); the defaults (allowlist, cn-or-san) are the safe starting point.
3. Add the devices you will issue to. Each **allowlist entry** is one serial number (or UDID). An allowlist entry is required in the default gate mode: an empty allowlist issues to nobody.
4. Point the MDM payload at the template's directory URL and deploy it to those devices.

Every change applies immediately to in flight orders. There is no restart and no cache to wait on.

Gate modes

A profile decides which devices reaching the template may enrol.

Mode	Behaviour
Allowlist (default)	Only devices whose serial or UDID is on the profile's allowlist may order. A device not on the list is refused. This is the fail closed default.
Open	Any device that passes attestation may order, with no allowlist check. This is an observation mode for a controlled fleet, not a setting to leave on in production.

Why allowlist is the default. Any genuine Apple device that can reach the template directory can attest its own real serial number. Without an allowlist, open mode would issue to any managed device that finds the URL. The allowlist is what limits issuance to the devices you decided on.

A **disabled** profile behaves exactly like no profile: the template stops offering device-attest-01 and device orders are refused invisibly. Disabling is the clean way to pause a template without deleting its allowlist.

Invisibly means the refusal names no device concept and reads exactly like the refusal for an identifier type the server does not know, whatever the client sends: a well formed serial, an empty value, an oversize one, and a device order mixed with a dns one all get the same answer. What that hides is whether **this template** takes device orders. It does not hide that the product implements the draft, which the version and this page already say.

CSR identifier binding

The Apple ACME payload cannot put the device identifier where the draft's strict mode expects it. Apple CSRs carry only RFC822, DNS, and URI subject alternative names, never the RFC 4043 PermanentIdentifier otherName. Deployed practice is to put the serial in the subject common name instead. The binding mode chooses how strict the server is about where the identifier appears in the CSR.

Mode	The CSR must carry the identifier as
cn-or-san (default)	the subject common name or a PermanentIdentifier SAN. This is the mode Apple clients work with.

Mode	The CSR must carry the identifier as
san-required	a PermanentIdentifier SAN only. Strict draft mode. Apple CSRs cannot satisfy this; use it only for clients that emit the otherName.
none	the CSR need not carry the identifier at all (privacy mode). The attestation still binds the device; the certificate simply does not restate the serial.

Two checks always run and are not configurable: the CSR public key must equal the attested key, and any dns SAN on a device CSR is refused. A wrong or unexpected value anywhere always refuses rather than issuing something loosely matched.

The Phase 0 lab probe confirmed ADCS passes a PermanentIdentifier otherName from the client CSR into the issued certificate untouched on an enrollee-supplies-subject template, with no CA wide flags. So san-required is honest on ADCS for a client that can produce the otherName; Apple clients stay on cn-or-san because their CSRs cannot. **Do not** enable the CA wide EDITF_ATTRIBUTESUBJECTALTNAME2 flag to work around this; it is a well known CA security hazard and is never required here.

ADCS template requirements

The template a profile points at is issued through the same ADCS path as any other ACME certificate, so it must:

- **Allow the enrollee to supply the subject.** The device identity lives in the CSR (the CN or the PermanentIdentifier SAN). A template that builds the subject from Active Directory instead will not carry the device identifier.
- **Carry the Client Authentication EKU** (1.3.6.1.5.5.7.3.2) if the certificate is for device authentication, plus any other EKU your use needs.
- **Be published on the CA and ACME enabled** in Ducks in a Row, the same as a template used for dns issuance.
- **Match the device key.** Apple hardware bound keys are ECDSA on the Secure Enclave (P-256). The template's key requirements must accept that key, or ADCS refuses at finalize with Denied by Policy Module.

The Apple MDM payload

Managed Device Attestation is driven by an ACME certificate payload (com.apple.security.acme) that your MDM delivers to the device. The settings that matter for attestation are:

Payload setting	Value
Directory URL	the template's directory, https://<server>/acme/<template>/directory
Client Identifier	the device serial number (what you put on the allowlist)

Payload setting	Value
Attest	true (request an attestation)
Hardware Bound	true (generate the key in the Secure Enclave)
Key Type	ECSECPriMeRandom (the Secure Enclave key type)
Key Size	256

Use your MDM's own interface or Apple's configuration profile reference for the exact key names and capitalisation; MDM vendors surface these differently. `Attest` and `Hardware Bound` must both be on, or the device produces an ordinary key with no attestation and the challenge cannot be satisfied.

The `Client Identifier` the payload sets becomes the order's permanent-identifier and must match an allowlist entry exactly.

External account binding does not apply

EAB (external account binding) cannot gate Apple device orders, because **the Apple ACME payload has no place for EAB credentials**. A device managed by your MDM cannot present an HMAC key id.

This has a concrete consequence: if EAB enforcement is set to **Required**, Apple devices cannot register an account and cannot enrol at all. That is expected, not a bug. The device attestation allowlist, not EAB, is the gate for device orders. The mode applies to the whole server rather than to one template, so putting devices on a template of their own does not help: keep EAB at Off or Optional wherever device attestation is in use. An account a device registered before the mode was raised keeps working, but any new registration is refused, a device enrolling again included.

Renewal and Apple rate limits

Apple rate limits fresh attestations per device. A device that attests too often in a short window is told to wait, so treat each attestation as a scarce resource:

- **Prefer longer certificate lifetimes.** Aim for 90 days or more so renewals are infrequent. A short lifetime that forces daily reattestation will hit Apple's limit.
- **The server never retries a spent attestation.** An attestation object is consumed once. Attestation failures are terminal for that challenge, not transient, so there is no server side retry loop burning through the device's budget.
- **The gate runs before the attestation.** A device that is not on the allowlist is refused at `newOrder`, before it spends an attestation, so a misconfigured fleet does not exhaust its budget against a closed gate.

Trust anchors and Apple root rotation

The Apple Enterprise Attestation Root CA ships embedded in Ducks in a Row, pinned by SHA-256 fingerprint and verified at load. It is shown on the trust anchor panel as a read only built in anchor. Verification fails closed if the embedded copy ever fails its pin.

Custom trust anchors are **additive**. You can add an anchor (by pasting its PEM on the trust anchor panel), but a custom anchor can never shadow, disable, or override the pinned Apple root. Adding an anchor makes the server trust attestations that chain to it, so only an administrator can add or remove one, and a certificate that equals a built in root is refused rather than stored as a second copy.

Two things custom anchors are for:

- **A test or synthetic attestation authority**, so a lab rig can drive the flow without a real Apple device.
- **Bridging an Apple root rotation**. When Apple rotates its attestation root:
 1. Obtain the new root and add it as a custom trust anchor. Devices attesting under either root now verify.
 2. Update to the Ducks in a Row release that pins the new root as the built in anchor.
 3. Remove the custom anchor once every device has moved to the new root.

Where refusals show up

A device order refused by the gate is recorded and surfaced on the dashboard activity feed as **blocked by the device attestation policy**, distinct from a domain allow list or EAB namespace refusal so you know to look at the device attestation card. Successful `device-attest-01` validations are counted in the Validation Methods widget alongside the `dns` challenge types.

11. Revocation

Ducks in a Row can revoke a certificate from the dashboard, and an ACME client can revoke its own. What the dashboard may touch is deliberately narrow, and it is narrowed by two independent things: a ceiling you cannot raise, and a scope you choose.

The ceiling you cannot raise

Before any setting is consulted, a certificate must clear a capability ceiling. It is revocable from the dashboard only when all of these hold:

- It has an extended key usage extension, and that extension is **not empty**.
- Every purpose in it is server authentication or client authentication, and nothing else.
- Its key usage does not include certificate signing or CRL signing.
- It is not a CA certificate.

A certificate whose capability cannot be determined at all, because there is no stored copy of it and no parsed detail, is refused rather than allowed. The ceiling fails closed.

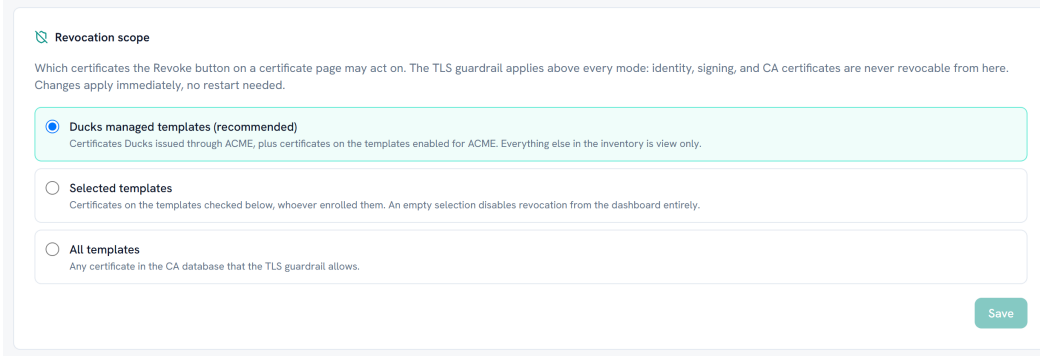
No administrator setting widens this. There is no configuration key, no break glass flag, and no dashboard control that lets the ceiling through. It is a subset rule, not a blacklist: a purpose nobody anticipated is outside the allowed set by construction, so it is refused without anyone having to think of it in advance.

The reason is blunt. A tool that proxies certificate requests should not be able to revoke a domain controller certificate, a code signing certificate, or the CA's own certificate. Those are exactly the revocations that take an organisation down, and none of them is a TLS certificate.

A certificate with no EKU at all is refused. An empty or absent EKU extension means valid for every purpose, which is the opposite of a narrow TLS certificate. It reads as unlimited, so the ceiling treats it that way.

The scope you choose

Underneath the ceiling, `revocationScope` decides which of the remaining certificates the dashboard offers to revoke. Set it on the Settings page.



Revocation scope

Which certificates the Revoke button on a certificate page may act on. The TLS guardrail applies above every mode: identity, signing, and CA certificates are never revocable from here. Changes apply immediately, no restart needed.

☒ **Ducks managed templates (recommended)**
Certificates Ducks issued through ACME, plus certificates on the templates enabled for ACME. Everything else in the inventory is view only.

☐ **Selected templates**
Certificates on the templates checked below, whoever enrolled them. An empty selection disables revocation from the dashboard entirely.

☐ **All templates**
Any certificate in the CA database that the TLS guardrail allows.

Save

The Revocation scope card on the Settings page, showing the three scope modes with Ducks managed selected

Mode	What the dashboard may revoke
ducks-managed	The default. Certificates Ducks in a Row issued, plus anything on a template you enabled for ACME
custom	Only certificates on the templates you list
all	Anything that clears the ceiling

Ducks managed is a union of two tests, and either one is enough: the certificate has an ACME record bridging it to a request this server made, or its template is one you enabled for ACME. The first covers certificates this server obtained. The second covers certificates on the same templates that something else obtained, which is usually what an administrator means by "ours".

custom with an empty template list disables dashboard revocation entirely. That is a legitimate configuration, and the startup log says so plainly rather than leaving you to wonder.

all is logged as a warning at every start. It is not forbidden, but it is worth being deliberate about.

Template names are matched in both forms. A template can be recorded under its programmatic name or its display name, so the match tries the stored form first and then maps display name to programmatic name through the list the CA publishes. If the CA is unreachable, only the raw match is available, so the scope narrows rather than widens. That is the safe direction.

`Certus:Acme:ExposeAllTemplates` does **not** widen revocation. The scope reads the recorded template selection directly rather than going through the ACME exposure policy, precisely so that an ACME break glass stays an ACME break glass and does not quietly become a revocation one.

What the CA has to allow

Dashboard revocation needs the server's machine account to hold **Issue and Manage Certificates** on the CA, which is a higher privilege than the **Read** that the inventory needs and the **Enroll** that issuance needs.

Granting it is a real decision. Without it, everything else keeps working and revocation returns a permission error naming the right that is missing. If you do not intend to revoke from the dashboard, do not grant it: the ceiling and the scope are defence in depth on top of the CA's own permissions, not a replacement for them.

ACME clients revoking their own certificates

An ACME client can revoke a certificate it holds through the standard `revoke-cert` endpoint, authorised the way RFC 8555 section 7.6 requires: either from the account that ordered it, or by proving possession of the certificate's own private key.

This path is not subject to the scope modes. A client revoking its own certificate is responding to something it knows about, usually a key compromise, and that must not wait on a dashboard setting an administrator chose months earlier. It only ever reaches certificates Ducks in a Row issued, so the blast radius is already bounded.

The one automatic revocation

There is exactly one place where the product revokes a certificate without being asked. At finalize, the certificate the CA returns is checked against the capability ceiling before it is handed to the client. A leaf that breaches the ceiling is revoked immediately, with reason code 5, and the order fails.

That is the guarantee behind the advisory check at new order time: a template's recorded metadata is unverified and might be wrong, but the certificate itself is checked every time and cannot be wrong. If a template is misconfigured so badly that it mints something outside the ceiling, the client never receives it.

Nothing else revokes on its own. In particular, an order that is abandoned or expires while the CA is holding it does not revoke anything: it logs the CA request id at warning level and leaves the decision to an operator. Automatic revocation of something a human might still want is the more dangerous mistake.

Where refusals show up

Every refusal, whether by the ceiling or by the scope, is recorded on the dashboard activity feed with the reason. If a certificate you expected to be revocable is not offered, that feed says which of the two stopped it.

12. Expiry Alerts

Ducks in a Row watches every certificate in its inventory and tells you before one expires, by email or by webhook.

Thresholds

By default an alert fires 30, 14, 7 and 1 days before expiry. Each certificate gets at most one alert per threshold, so a certificate does not nag you hourly for a month. The monitor sweeps every 60 minutes.

Both are configurable: `Certus:Alerts:ThresholdDays` and `Certus:Alerts:CheckIntervalMinutes`. See [configuration](#).

Revocation lists

Ducks in a Row also watches the certificate revocation lists of every CA in the chain of the connected CA, including the ones published by a CA above it.

This matters more than it sounds. A certificate expiring breaks one service. A CRL expiring breaks revocation checking for **everything that CA ever signed**, at once, on every client that checks revocation: domain controllers, VPN and wireless authentication, and the certificate authority's own service. An offline root's CRL is the dangerous one, because it expires on a schedule of its own, nothing automatic replaces it, and the root is switched off and easy to forget.

Where each CRL is read from

- The connected CA's own CRLs come from the CA itself, over the same connection everything else uses, and from the locations the CA tells its clients to read them.
- The CRL covering the CA's own certificate, which on a two tier estate is the offline root's, is read from the distribution points named on that certificate. Both `ldap://` and `http://` locations are read.

Every copy is checked against the certificate of the CA that should have signed it, and one that does not verify is ignored rather than trusted.

Each copy is watched separately, and that is the point. A root CRL renewal that is published to the directory but never copied to the web server leaves the web server serving a CRL that expires, and every client that reads the HTTP location fails while the domain looks healthy. Ducks in a Row warns about the stale copy, names the location, and says that a newer CRL already exists elsewhere.

When it warns

Which rule applies depends on who replaces the CRL:

The CRL is	What you get
Replaced by a CA on a timer (the connected CA's own, and any short lived one)	One warning when the CA misses its own scheduled publication, and one when the CRL actually expires

The CRL is	What you get
Published by hand (an offline root or policy CA, recognised by a CRL that outlives your widest threshold)	The same ladder as certificate expiry: 30, 14, 7 and 1 days by default, then one when it expires

The ladder deliberately does not run against a CRL a CA replaces on a timer. A Windows CA publishes a weekly CRL that is valid for about seven and a half days and replaces it with about twelve hours to spare, so the ordinary thresholds would fire several times a week on a CA doing exactly what it should.

A warning fires once per CRL. Publishing a new one resets the ladder, so a root CRL renewed in good time goes quiet again on its own.

When a CRL cannot be read

A distribution point that cannot be reached, or a CA that is down, does not silence anything: the warnings continue against the last copy that was read, and say when that was. An unreachable location is not evidence that the CRL behind it was renewed.

The **Revocation lists** card on the Settings page shows every CRL being watched, each place it is published, and what was last read from each.

Configuration

```
{
  "Certus:Alerts": {
    "Crl": {
      "Enabled": true,
      "CheckIntervalMinutes": 60,
      "FetchTimeoutSeconds": 15,
      "MaxCrlBytes": 33554432
    }
  }
}
```

The warning thresholds are the ones above, shared with certificate expiry: a site that wants sixty days of notice wants it for both.

Email

The whole SMTP transport can be set from the **Alerts** card on the Settings page: relay host, port, transport security, username and password, sender address and name, and the recipient list. That is the recommended route. It needs no file editing, and the password is stored encrypted in the data folder rather than sitting in a configuration file in the clear.

The file form remains available for scripted or unattended setup:

```
{
  "Certus:Alerts": {
    "Enabled": true,
```

```
"CheckIntervalMinutes": 60,
"ThresholdDays": [30, 14, 7, 1],
"Smtp": {
  "Host": "smtp.yourdomain.local",
  "Port": 587,
  "TlsMode": "starttls",
  "Username": "ducks@yourdomain.local",
  "Password": "your-smtp-password",
  "FromAddress": "ducks@yourdomain.local",
  "FromName": "Ducks in a Row",
  "Recipients": ["admin@yourdomain.local"]
}
}
```

TlsMode accepts none, starttls or implicit. Leave it out and the port decides: 465 means implicit TLS, anything else negotiates mandatory STARTTLS.

A null Smtp block means email alerts are off. That is the shipped default.

Saving from the dashboard changes who owns the setting. The Alerts card writes into the settings file in the data folder, which outranks appsettings.json. After that, editing those keys in appsettings.json has no effect. The card lists the keys it now owns, so the situation is visible rather than puzzling. An environment variable or command line argument still outranks both, and where one is in use the card disables that field and says why instead of accepting an edit that could never take effect.

A password saved from the dashboard is protected with the machine's Data Protection keyring. It never appears in any file in readable form and is never returned by the API. Restoring the data folder onto a different machine makes it undecryptable by design, so save it again on the new machine. See [backup and restore](#).

Webhooks

A webhook alert is an HTTP POST carrying a JSON payload, signed with HMAC-SHA256 in the X-Certus-Signature header so the receiver can verify it came from you.

```
{
  "Certus:Alerts": {
    "Webhook": {
      "Url": "https://hooks.example.com/services/YOUR/WEBHOOK/URL",
      "Secret": "your-hmac-secret",
      "Headers": {
        "X-Custom-Header": "value"
      }
    }
  }
}
```

A null webhook block means webhook alerts are off, which is the shipped default.

Every alert carries an event name, so a receiver can route on it:

Event	Sent when
<code>certificate.expiring</code>	One or more inventory certificates crossed a threshold
<code>crl.expiring</code>	A revocation list published by hand crossed a threshold
<code>crl.overdue</code>	A CA did not replace its own revocation list on schedule
<code>crl.expired</code>	A revocation list is past its next update
<code>server.certificate.renewal_failed</code>	Automatic renewal of this server's own certificate failed
<code>test.alert</code>	The test button on the Alerts card

A CRL payload names the issuing CA, whether it is a base or a delta CRL, its number, when it expires, and every location it is served from. When a newer CRL is published somewhere else it carries that number too, which is what tells a receiver that a renewal reached one location and not another.

The webhook block is deliberately not editable from the dashboard. It is always read from the file and never written by the Settings page. An alert destination is a place your certificate inventory gets described to, so changing it is a file edit and a service restart, not a form field.

A saved change needs a restart

Alerting reads its settings once, when the service starts. A change saved on the Settings page is stored but not in force until you restart, and the card says so rather than pretending otherwise. It offers a restart button for exactly this.

Alert history

The Alerts card shows whether monitoring is on, how often it checks, the thresholds in force, who is on the recipient list, whether a webhook is configured, and the most recent alerts including any that failed to send. A failed send is worth looking at: an alert that could not be delivered is indistinguishable, from the outside, from a certificate that never needed one.

The same data is available from the API at `GET /api/alerts/history` and `GET /api/alerts/summary`.

Testing it

The Alerts card has a test send. Use it after any change to the transport: a relay that refuses the service's credentials will refuse them at 3am on the day a certificate expires just as readily as it does during a test, and the test is the cheaper way to find out.

13. Hardening

Ducks in a Row ships with defaults that favour first issuance on an internal network. This page collects the optional lockdowns and explains the reasoning behind each default, so an open posture is an informed choice rather than an accident.

The data folder

The installer restricts the data folder, `C:\ProgramData\Ducks in a Row` by default, to SYSTEM and Administrators, and lets Users only read and run the installation folder. Keep it that way. Whoever can write `settings.json` can name the group the dashboard admits, and whoever can write `ducks-setup.json` sets the ACME policy. The service refuses either file if anyone else may write it, which is a backstop, not a reason to widen the folder.

Challenge validation egress

For HTTP-01 and TLS-ALPN-01 challenges, the server itself is the party that connects out: it fetches `http://{host}/.well-known/acme-challenge/{token}` on port 80, or opens a TLS connection on port 443 and inspects the ALPN certificate. DNS-01 and device-attest-01 never connect to the validation target, so nothing in this section affects them.

Because the validator runs next to the CA, those outbound connections are worth fencing. Out of the box:

Target	Range	Default
Loopback	127.0.0.0/8, ::1	blocked
Link local, including the cloud metadata address 169.254.169.254	169.254.0.0/16, fe80::/10	blocked
IPv6 unique local	fc00::/7	blocked
The literal name localhost		blocked
RFC 1918 private ranges	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	allowed

Every address a name resolves to is vetted at connect time, so a record that flips between a public and a private address (DNS rebinding) does not slip through, and IPv4 addresses wrapped in IPv6 form are unwrapped before the check. Redirects are not followed by default, so a challenge server cannot bounce the validator to an internal address either.

Why the private ranges are open by default. An internal CA usually validates hosts that live on exactly 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16; blocking them by default would break the product's primary use case on day one. The cost of the open posture: an ACME client with an account on your server can point the validator at internal hosts on ports 80 and 443 and learn from the timing whether something answers there. Validation only succeeds on the

exact expected response, so nothing is read back, but it is an internal reachability oracle. Once nothing you validate lives on a private address, turn the flag below on and that oracle is gone.

Blocking the private ranges

If every name your CA validates resolves to a public address, set one flag in `settings.json` in the data folder (`C:\ProgramData\Ducks in a Row`):

```
"Certus:Acme:ChallengeValidation": {  
  "BlockPrivateRanges": true  
}
```

Or as an environment variable: `Certus__Acme__ChallengeValidation__BlockPrivateRanges=true`. Restart the service after either; the egress settings are read at startup.

With the flag on:

- An order that names a private IP address directly is refused at order time with `rejectedIdentifier`.
- A name that resolves to a private address fails HTTP-01 and TLS-ALPN-01 validation as a hard policy rejection. The challenge is marked invalid and is not retried, because nothing transient is wrong.

Blocking more ranges

`AdditionalBlockedCidrs` fences off anything else, in address/prefix form. The checks are additive: an address that any rule blocks is blocked. For example, to also cover the carrier grade NAT range:

```
"Certus:Acme:ChallengeValidation": {  
  "BlockPrivateRanges": true,  
  "AdditionalBlockedCidrs": ["100.64.0.0/10"]  
}
```

An invalid CIDR entry stops the service at startup rather than silently weakening the fence; the reason is in the log file under `C:\ProgramData\Ducks in a Row\logs`.

Settings reference

All keys live in the `Certus:Acme:ChallengeValidation` section.

Setting	Default	Purpose
<code>AllowRedirects</code>	false	Whether validation follows HTTP redirects. Leave off so a challenge server cannot redirect the validator to an internal address
<code>BlockLoopback</code>	true	Refuse loopback validation targets
<code>BlockLinkLocal</code>	true	Refuse link local targets, including the cloud metadata address
<code>BlockUniqueLocalIpv6</code>	true	Refuse IPv6 unique local targets
<code>BlockPrivateRanges</code>	false	Refuse the RFC 1918 private ranges. See above

Setting	Default	Purpose
AdditionalBlockedCidrs	[]	Extra address/prefix ranges to refuse

This screen applies to the validator's own outbound connections only. It is not a host firewall, and it does not constrain what the CA may issue; the allowed domain list does that.

Hardening the CA behind Ducks in a Row

Ducks in a Row is a proxy. It decides *who* may ask for a certificate and *for which names*, then hands the request to Active Directory Certificate Services, which decides what to issue. A hardened proxy in front of an unhardened CA still leaves you exposed, because anything with a domain account can talk to the CA directly and never touch Ducks in a Row at all.

Apply the July 2026 update

CVE-2026-54121, published as "CertiGhost", is a critical elevation of privilege in AD CS with a public proof of concept. A low privilege domain account with network reach to the CA can drive certificate enrollment into impersonating a domain controller, which leads to full domain compromise. Microsoft shipped the fix on 14 July 2026.

Install the July 2026 security update on every CA in the forest. Nothing in this product substitutes for that patch.

What Ducks in a Row does and does not do here. The attack rides in ADCS request attributes: `cdc` points the CA at an attacker controlled host and `rmd` names the domain controller to impersonate. Ducks in a Row sends exactly one request attribute, `CertificateTemplate`, and builds it in a single place that refuses any template name carrying a control character, a line separator, or a Unicode formatting character, so an ACME client cannot append a second attribute through this product. That closes Ducks in a Row as a route to the bug. It does not fix the bug, and it does not stop anyone reaching the CA by any other path.

Reduce the blast radius

Worth doing regardless of patch state, and all of it is CA and directory configuration rather than product settings:

Step	Why
Set <code>ms-DS-MachineAccountQuota</code> to 0	The default of 10 lets any domain user create machine accounts, which several AD CS escalation paths need. Grant machine join to a delegated group instead
Enable CA auditing with <code>AuditFilter</code> set to 127	Records every CA event, including the request attributes on each submission, so an attempt is visible afterwards. <code>certutil -setreg CA\AuditFilter 127</code> , then restart the CA service

Step	Why
Review Enroll and Autoenroll on every published template	Enroll rights held by broad groups such as Domain Users or Authenticated Users are what turn a template flaw into a domain wide one
Prefer templates that build the subject from Active Directory	An enrollee supplies subject template trusts the requester for the name. Ducks in a Row validates names before submitting, but a template that does not need that trust is a smaller target
Keep the CA off general purpose networks	The precondition for CertiGhost is network reach to the CA with a domain account. Restricting who can reach the enrollment endpoints shrinks the population that can try

Confirming your own exposure

The service account matters as much as the CA configuration. Ducks in a Row submits every request as its own computer account, so that account's Enroll rights are the ceiling on what any ACME client can obtain through it. Grant Enroll only on the templates you intend to expose over ACME, and check the enabled template list on the setup wizard against that grant.

14. Backup and Restore

Everything worth backing up is in one folder. One thing in it does not survive being moved to another machine, and knowing which is the whole of this page.

What to back up

The data folder, by default `C:\ProgramData\Ducks in a Row\`. If you chose a different one at install time it is recorded in the registry under `HKLM\SOFTWARE\Haruspex Systems\Ducks in a Row`, value `DataDirectory`.

Item	What it holds	Survives a move to another machine
<code>ducks.db</code>	ACME accounts, orders, certificate inventory, alert history, EAB credential records, device attestation profiles	Yes
<code>settings.json</code>	CA connection string, external URL, alert transport	Yes
<code>ducks-setup.json</code>	Enabled templates, allowed domains, EAB enforcement mode, revocation scope	Yes
<code>logs\</code>	Serilog daily files, 30 retained	Yes
<code>keys\</code>	The Data Protection keyring	No
<code>ducks-selfsigned.pfx</code>	The fallback HTTPS certificate	Regenerated if absent

The application folder under Program Files does not need backing up. Reinstalling replaces it, and it holds nothing you configured.

Taking a backup

Stop the service first, so the database is quiescent and the write ahead log has been folded back in:

```
Stop-Service DucksInARow
Copy-Item -Recurse "C:\ProgramData\Ducks in a Row" "D:\Backups\ducks-$(Get-Date -Format yyyyMMdd)"
Start-Service DucksInARow
```

If you cannot stop the service, back up the whole folder including the `ducks.db-wal` and `ducks.db-shm` files. Copying `ducks.db` on its own while the service is running gives you a database missing every change still in the write ahead log.

Stopping the service is the simpler answer, and it is quick.

A backup tool that cannot cope with the sidecar files can be accommodated by setting `Certus:EnableWalMode` to `false`, which converts the database to rollback journal mode on the next start and leaves a single file. See configuration.

Restoring onto the same machine

Copy the folder back and start the service. Everything works, including the encrypted secrets, because the Data Protection keyring came back with it and the machine it is tied to has not changed.

Copy it back as an administrator, so the restored files are owned by the Administrators group. The service ignores `settings.json` and `ducks-setup.json` if anyone else owns them or may write them; if the log says it has, see [Troubleshooting](#).

Restoring onto a different machine

Everything works **except anything that was encrypted**, because the keyring is tied to the machine that created it. On the new machine:

- **EAB credential secrets** cannot be verified. Clients get 403 unauthorized and the log tells you to regenerate.
- **An SMTP password saved from the dashboard** cannot be read, so email alerts do not send.

This is by design. It means a copy of your data folder is not, by itself, a copy of your credentials, and someone who obtains the backup does not thereby obtain the secrets in it.

Recovering takes two steps, and neither loses history:

1. On the ACME page, regenerate each EAB credential. The key id, name, expiry, namespace and owner are all stored in the clear and come across unchanged, so only the HMAC key needs redistributing. See [external account binding](#).
2. On the Settings page, enter the SMTP password again and save.

Everything else, including every issued certificate, every ACME account and the whole alert history, is intact.

Do not try to move the keyring. Copying `keys\` to the new machine does not help, because the files in it are themselves protected with the original machine's DPAPI key. Regenerating is the supported recovery, and it is quicker than any alternative.

What a backup does not protect

The certificates themselves live in the certificate authority's database, not in this one. Ducks in a Row keeps an inventory of them, not the authority.

So restoring a backup does not reissue, revoke or otherwise change a single certificate. It restores the record of them. Backing up the CA is a separate job and a more important one; if you have not got a CA backup, that is the gap worth closing first.

15. Troubleshooting

Where to look first:

- **Logs:** logs\ducks-*.log in the data folder (default C:\ProgramData\Ducks in a Row; daily files, 30 day retention).
- **Windows Event Log:** Application channel, source DucksInARow.Service.
- **Health:** Invoke-RestMethod https://your-server:5001/health returns Healthy, Degraded, or Unhealthy.
- **Service:** Get-Service DucksInARow.

Health result	Meaning
Healthy	Database is accessible and the CA connection works
Degraded	Database works but the CA is not reachable
Unhealthy	Database is not accessible

CA operations answer 503, or the dashboard sends me to the setup wizard

No CA is configured yet. The service starts unconfigured on a fresh install: it serves the setup wizard and answers 503 for every CA operation until the wizard connects a CA. Complete the wizard at <http://localhost:5000/setup>; it discovers the CAs in Active Directory, tests the connection, checks the service's rights on the CA, writes the configuration to settings.json in the data folder, and restarts the service.

If the wizard says setup is already complete but the dashboard still reports the CA unavailable, check settings.json in the data folder for Certus:CaConnectionString, then restart:

```
Restart-Service DucksInARow
```

DISP_E_MEMBERNOTFOUND (0x80020003) in the logs

The ADCS COM classes are not registered on the Ducks in a Row server. Install the ADCS Remote Administration Tools and restart the service. This feature is required even though this server is not the CA:

```
Install-WindowsFeature RSAT-ADCS-Mgmt
Restart-Service DucksInARow
```

Health is Degraded (CA not reachable)

- Confirm Certus:CaConnectionString (in settings.json in the data folder) is in CAHOST\CA Name form.

- Confirm the server is domain joined and can reach the CA.
- Confirm DCOM and RPC are allowed to the CA: TCP 135 plus the dynamic range 49152 to 65535.
- Test from the Ducks in a Row server:

```
certutil -config "CA01.corp.example.com\Corp Issuing CA" -ping
```

The dashboard is empty

- The machine account needs **Read** permission on the CA (Certification Authority console, CA properties, Security tab). Enrolment can work without it, but the inventory sync cannot read the CA database. The log reports "CA view access denied" in that case.
- The first sync may not have run yet. Sync runs every `Certus:SyncIntervalMinutes` (default 5). Restart the service to force one.

The rights check shows Failed or Unproven

The wizard and the **Service rights on the CA** card on the Settings page check what the service's own account may do. [Verifying](#) explains the five statuses. The rows that come up most:

- **Reaching the CA as this server: Failed, "the CA answered and refused"**. The account lacks **Request Certificates** on the CA. Authenticated Users hold it on a default CA, so someone has taken it away. A CA that refuses remote requests altogether (the `IF_NOREMOTEICERTREQUEST` interface flag) answers the same way. The template list, the CA certificate downloads and CRL watching read through the same interface, so they fail too.
- **The certificate view the inventory reads: Failed**. Grant **Read** on the CA. Request Certificates does not open the view. If the row says the CA reports Read and still refused the view, the CA may refuse remote administration altogether (the `IF_NOREMOTEICERTADMIN` interface flag).
- **Issue and Manage Certificates: Not granted (optional)**. Only revocation needs it. Everything else works without it; see [revocation](#) before granting it.
- **Issue and Manage Certificates: Unproven, "the CA would not report the service's roles"**. The CA reports roles only to an account holding Read or more. Grant Read, then **Check again**.
- **Enroll on a template: Failed**. Either a deny entry refuses it, or no entry grants it to the account or to any group it belongs to. See [certificate template permissions](#). A grant to a group from another domain of the forest is not visible from the service's server, so a row that says "no grant" can be wrong in a multi domain forest. The first enrolment settles it.
- **Enroll on a template: Inferred**. Not a problem. The permissions grant it, and nothing has enrolled from that template yet.
- **Any row: Unproven, "did not answer within N seconds"**. The CA or a domain controller was slow or unreachable. Check again, and read the service log.

A certificate request fails with "unknown certificate template"

The template name in the directory URL must match a template the CA publishes, either by its programmatic name (AD cn) or its display name. Names are matched case insensitively. Check the exact names in the setup wizard's template list. Also confirm the template is published to Active Directory and the machine account has **Enroll** on it.

The display name must not carry a control character, a Unicode line separator, or a formatting character. See the next section if the client reports a 400 rather than a 404.

The directory URL answers 400: the URL contains a formatting character

The client reports an error like:

```
urn:ietf:params:acme:error:malformed
The request URL contains a formatting character (U+00AD) at position 9, so it
was refused before routing.
```

Cause. The template's display name carries an invisible character. U+00AD SOFT HYPHEN is the usual one: a word processor inserts them at hyphenation points, and a display name is routinely typed by pasting. U+200B ZERO WIDTH SPACE and the bidirectional overrides do the same thing. These characters are invisible in the Certificate Templates console, on the dashboard, and in the client's own configuration file alike, so the name looks correct everywhere you would go to check it.

The server refuses any URL carrying one before routing, because such a character can forge or disguise a line in the service log, which is the record [hardening.md](#) tells you to read after a suspected enrollment attack. Nothing about the template itself is wrong: it still issues normally.

Two fixes, either one works.

- Address the template by its **programmatic name** (its AD cn), which never carries these characters in practice. Use `/acme/WebServer/directory` rather than `/acme/Web Server/directory`. This needs no change on the CA.
- Retype the **Template display name** on the **General** tab of the template in the Certificate Templates console. Retype it rather than paste it, or the same character comes back. No republish is needed; the template list is re-read every five minutes.

Where this is reported. The setup wizard's template step flags the selected template under "ACME readiness", naming the code point and the position. The service log carries one warning near startup for any affected template that is exposed over ACME. Neither ever prints the name itself, for the same reason the URL is refused: writing the name would put those characters into the log.

If the programmatic name is the one carrying it, the template cannot be used at all, not over ACME and not from the settings page, because Ducks in a Row will not build an ADCS request attribute string from such a name. The setup wizard hides such a template and counts it in its hidden templates note. A programmatic name is fixed when a template is created, so the fix is to duplicate the template with a clean name and publish the copy.

If the template OID is the one carrying it, nothing is refused. The wizard notes it on the template's "ACME readiness" list, and the startup warning counts it when the template is exposed over ACME, but the template issues certificates and is addressed exactly as any other, because nothing in Ducks in a Row routes on the OID or builds a request from it. It is reported because the wizard prints the OID on every template row to identify the template, and the value you see there is not the value the CA stored: the character is invisible, so the OID looks correct on that row, in the Certificate Templates console, and anywhere else it is shown. The row itself is safe to read. Each of the three values on it is isolated, so a bidirectional override inside one can no longer reorder the text around it. The OID is set when a template is created, like the programmatic name, so there is no field to retype: either ignore the note, which costs nothing, or duplicate the template if you would rather the row read as it is stored.

The directory URL answers 403: template "is not enabled for ACME"

The template exists on the CA but is not exposed over ACME. Exposure fails closed: only templates enabled in the setup wizard are served. Common causes:

- **The template was not selected in the wizard.** Re-run the wizard, or check which template it recorded, and address that template in the directory URL.
- **The wizard has not recorded a template selection yet.** A fresh install exposes nothing over ACME until the wizard's template step records a choice; every template answers 403 before that.
- **The wizard status file is unreadable.** If `ducks-setup.json` in the data folder is corrupt, the service log carries an Error naming the file, and no templates are exposed until the file is fixed or the wizard is re-run.

Setting `Certus:Acme:ExposeAllTemplates` to `true` exposes every CA published template regardless of the wizard's selection. That is a break glass override, named in a startup warning; leave it off in normal operation.

An ACME order is rejected: domain is not in the allowed domain list

The client reports an error like:

```
urn:ietf:params:acme:error:rejectedIdentifier
This server's domain policy does not allow issuance for: web.other.local.
An administrator can change the allowed domains on the Settings page.
```

The allowed domain restriction is on and the requested name falls outside every allowed domain. On the dashboard, open **Settings, Allowed Domains** and add the domain (subdomains of an entry are covered automatically), or turn the restriction off. Changes apply immediately, no restart needed. Each rejected order also appears on the dashboard activity feed as **Rejected**.

An ACME order is rejected: outside the credential's domain namespace

The client reports an error like:

```
urn:ietf:params:acme:error:rejectedIdentifier
The external account credential 'web team' this account registered with does
```

```
not allow issuance for: other.example. An administrator can change the
credential's domain namespace on the ACME page.
```

The account is bound to an EAB credential whose domain namespace does not cover the requested name. On the dashboard, open **ACME**, find the credential, and use **Edit** to widen its namespace (or empty the list to remove the restriction). Changes apply to the next order immediately. When the Settings allowed domain list is also on, it stays the ceiling: a namespace entry outside it does not allow issuance there. These rejections appear on the activity feed labeled as blocked by the credential's namespace, so they are distinguishable from allowed domain list refusals.

Registration is refused: externalAccountRequired

```
urn:ietf:params:acme:error:externalAccountRequired
```

The enforcement mode on the ACME page is **Required** and the client tried to register without an external account binding. Create a credential on the ACME page (or use an existing one), hand its key id and HMAC key to the client, and pass them with the client's EAB flags; the credential row's **Client setup** action shows the exact command for each supported client, and [Connecting ACME clients](#) lists the flags. Accounts that registered before the mode became Required are not affected; only new registrations need a credential.

Registration or ordering fails with 403 unauthorized naming a credential

The message names the cause:

- **Unknown external account key identifier**: the kid was mistyped, or the credential was deleted from a different install than the client points at. Check the key id against the ACME page.
- **The credential has been revoked**: revocation is terminal and also suspends orders from accounts already bound to it. Create a new credential and re register the client with it.
- **The credential expired on ...**: extend or clear the expiry with the credential's **Edit** action to put it and its accounts back in use, or create a new credential.
- **Signature verification failed**: the client holds a stale secret, most often after a **Regenerate** (every old copy stops verifying the moment the secret rotates). Paste the current secret into the client; if it was never saved, regenerate again and use the new value.
- **The credential is unusable on this server**: see the next section.

EAB secrets stopped verifying after moving or restoring the data folder

The MAC secrets are encrypted at rest with a Data Protection keyring stored next to the database and protected by the machine's DPAPI. Restoring the data folder onto a different machine (or reinstalling Windows) means the keyring can no longer be decrypted, so every stored EAB secret becomes unreadable. Verification fails closed: clients get 403 unauthorized and the log says the stored secret could not be decrypted.

The credentials themselves survive: the key id, name, expiry, and domain namespace are plain columns. The recovery is **Regenerate** on each credential from the ACME page, which issues a new secret on the same key id; hand the new secret to the client and nothing else changes. Already registered accounts are unaffected (the secret is only used at registration).

The dashboard will not let me revoke a certificate

Two independent things can stop it, and the dashboard activity feed says which.

Cause: the capability ceiling. A certificate is revocable from the dashboard only when its extended key usage is present, non empty, and limited to server authentication and client authentication, its key usage carries no certificate or CRL signing, and it is not a CA certificate. A certificate whose capability cannot be read at all is refused as well.

No setting raises this. It is what stops the product from being able to revoke a domain controller certificate or the CA's own certificate, so the fix is not to widen it. Revoke that certificate from the Certification Authority console instead.

Cause: the revocation scope. Underneath the ceiling, the `revocationScope` setting decides which certificates are in reach. The default, `ducks-managed`, covers certificates Ducks in a Row obtained plus anything on a template you enabled for ACME. If the certificate is on some other template, widen the scope on the Settings page, or revoke it at the CA.

A `custom` scope with an empty template list disables dashboard revocation completely. The startup log says so; check there before assuming something is broken.

Cause: the CA has not granted the right. Revocation needs the server's machine account to hold **Issue and Manage Certificates** on the CA, which is separate from the `Read` that the inventory needs. The error names the missing right. See [revocation](#).

An ACME client revoking its own certificate is not subject to the scope modes, so a client can still revoke through `revoke-cert` when the dashboard will not.

A device order is refused, or the device challenge never appears

`device-attest-01` fails closed, and the two refusals look different on purpose.

Cause: no profile, or a disabled one. A template with no device attestation profile does not offer `device-attest-01` at all, and a device order is refused as `unsupportedIdentifier`, exactly as if the feature did not exist. That invisibility is deliberate: it means an install that does not use device attestation gives nothing away about it. Create and enable a profile for the template on the ACME page.

Cause: the device is not on the allowlist. This one is visible. The order is refused with `rejectedIdentifier` and a subproblem naming the device, and the refusal appears on the activity feed labelled as a device attestation policy refusal. Add the device's permanent identifier to the profile's allowlist. An empty allowlist issues to nobody.

Cause: the attestation did not verify. The only supported format is apple, and the leaf must chain to the pinned Apple Enterprise Attestation root or to a trust anchor you added. A custom anchor is additive and can never replace the built in root.

Cause: the CSR does not match. The key in the certificate request must be the key the attestation vouches for, with no exception and no setting. A device CSR carrying any DNS subject alternative name is always refused.

The gate is re-checked when the order is created, when the challenge is validated, and at finalize, so disabling a profile mid flight stops an order that was already moving. See [device attestation](#).

An ACME client reports a TLS or certificate trust error

The HTTPS endpoint uses a self signed certificate by default (ducks-selfsigned.pfx in the data folder). ACME clients reject untrusted TLS on the ACME server. Either:

- give Ducks in a Row a certificate your clients trust (Kestrel HTTPS certificate in appsettings.json), or
- add that certificate's root to each client's trust store, or
- for a quick lab test, use the per client switch to skip the check (see [Connecting ACME clients](#)).

The dashboard says "Server certificate renewed. Restart to apply it"

Ducks in a Row renews the certificate it serves for its own HTTPS endpoint before it expires. The renewal deliberately does not restart the service: a production service must not bounce itself unannounced, and every ACME client connected at that moment would lose its connection.

So the new certificate is enrolled, installed into LocalMachine\My, and recorded in the settings overlay, and the service keeps serving the previous certificate until it restarts. Renewal runs a full window ahead of expiry, so there is no hurry, but the notice stays until you apply it and grows more urgent as the certificate actually in use runs down.

Apply it either way:

- press **Apply now** in the banner, or **Restart now to apply it** in the HTTPS certificate section of the Settings page, or
- restart the service yourself:

```
Restart-Service DucksInARow
```

The superseded certificate stays in the machine store until the restart has happened, because removing it would delete the private key the running service is still using. It is cleaned up automatically on the next renewal check.

The dashboard says automatic renewal of the server certificate failed

Nothing was changed: the service is still serving its current certificate, and the attempt is retried on the next check (daily by default). The Settings page carries the reason the certificate authority gave. The usual ones:

- **denied** — the computer account lost Enroll permission on the template. Grant it on the Security tab of that template.
- **pending** — someone turned on "CA certificate manager approval" for the template, so the CA is holding the request for a person to approve. The server's own certificate is renewed on a schedule and does not wait around for that, so either approve the queued request in the Certification Authority console, or turn the setting off on the Issuance Requirements tab and deny the queued request. ACME issuance on that template is not stopped by this; see the section below for what it does instead.
- **sanMismatch** — the template stopped honouring the requested subject, so the issued certificate no longer covers the external URL host. Set the Subject Name tab back to "Supply in the request". The certificate the CA issued was discarded rather than applied.
- **failed** — usually the CA being unreachable; see the CA connectivity entries above.

If email or webhook alerts are configured, one alert is sent per failed attempt. The server's own certificate is deliberately excluded from the ordinary 30, 14, 7, and 1 day expiry alerts, because the product renews it itself; this failure alert is the only thing those channels say about it.

To turn automatic renewal off, or to change the window, set

`Certus:HttpsCertificateAutoRenewalEnabled` Or `Certus:HttpsCertificateRenewalWindowDays` in `appsettings.json`. The default is on, 30 days, capped at a third of the certificate's own validity so a short lived template does not renew on every check.

An ACME client waits a long time, and the order stays "processing"

The template has "CA certificate manager approval" turned on, on the Issuance Requirements tab. Every request against such a template is held by the CA until a person approves it in the Certification Authority console, under **Pending Requests**. The setup wizard's readiness checklist flags this when you pick the template, and it is a legitimate way to run a CA, so nothing stops you using it.

What happens meanwhile:

- The order sits in `processing`, which is the correct ACME status for an issuance the server has accepted and not yet completed.
- The service re-checks every held request once a minute. When you approve one, the certificate is collected and the order goes to `valid` on the next check, with no action from the client beyond continuing to poll.
- If you deny the request, the order goes to `invalid` and the client is told the CA refused it.
- If nobody decides either way before the order expires, seven days after it was created, the order goes to `invalid` and the log names the CA request id. **Approving that request afterwards issues a certificate that no longer has an order behind it**, so deny it instead, or revoke what it issues.

Most ACME clients give up waiting long before a person gets to the console. That is a client timeout, not a failure of the order: the order completes regardless, and the client collects the certificate the next time it runs. Clients that renew on a timer handle this by themselves. If you want issuance to complete inside a single client run, the template has to issue without approval.

To change how often held requests are re-checked, set `Certus:Acme:PendingIssuance:PollIntervalSeconds` in `appsettings.json`. The default is 60, and values are clamped to between 5 seconds and an hour.

An ACME client gets "connection refused"

- The service is running: `Get-Service DucksInARow`.
- The firewall rules exist:

```
Get-NetFirewallRule -DisplayName "Ducks in a Row Certificate Proxy"
```

- The client reaches the server on the right port (5001 for HTTPS, 5000 for HTTP).

The service ignores a configuration file

The log has a Critical line saying that `settings.json` or `ducks-setup.json` "is being ignored because it cannot be trusted". Then either the dashboard sends you to the setup wizard, or every ACME request is refused because no template is enabled.

The service reads these two files only when no one but SYSTEM and Administrators could have written them:

- the file must be owned by SYSTEM or the Administrators group;
- no one else may have write access to it.

The log line says which rule the file broke. A file anyone else could change is treated as if it were not there, because the dashboard's administrator group and the ACME policy live in these files.

A file in this state was changed by someone other than an administrator, or its permissions were widened by hand. **Check what it contains before you trust it again.** Then either:

- delete it and run the setup wizard again, which writes a fresh one; or
- in an elevated PowerShell, give it back to Administrators, reset its permissions to the folder's, and restart the service:

```
$file = "C:\ProgramData\Ducks in a Row\settings.json"
icacls $file /setowner "*S-1-5-32-544"
icacls $file /reset
Restart-Service DucksInARow
```

Saving from the setup wizard or the Settings page also replaces `settings.json` with a fresh file. It keeps only what that page saves, so add anything else you had in it again.

The service will not start

- Check the log files and the Application event log.
- Validate `appsettings.json` for JSON errors (a trailing comma or missing quote will stop startup).
- Confirm ports 5000 and 5001 are not already in use.

- Run the executable directly from an elevated prompt to see the error:

```
& "C:\Program Files\Ducks in a Row\DucksInARow.Service.exe"
```

That last step matters more than it looks. A service that crashes before its logger starts hides its own error: Windows reports a start timeout, which reads like a slow service, and the event log reports a crash in `coreclr.dll`, which reads like a product bug. Only the standalone run prints the real reason.

The installer fails with Error 1920

The full text is "Verify that you have sufficient privileges to start system services", and it almost never means that. The installer is already elevated. The message appears because the service started and then exited before it could signal Windows, so this is the same problem as the section above: read `logs\ducks-*.log` in the data folder first.

A blank or missing log means the crash happened before the logger started.

On Windows Server 2022 the usual cause is an outdated servicing level. The .NET 10 runtime requires Control-flow Enforcement Technology (CET, shadow stacks) on Server 2022, and an installation that has not been patched since early 2022 does not provide it. The process faults inside `coreclr.dll` with exception code `0x80131506`, and running the executable directly prints the actual message:

```
Fatal error. Your Windows doesn't fully support CET. Please install all available Windows updates.
```

Build 20348.587 is confirmed too old. Check yours with `winver`. Install current Windows updates and retry. Server 2019 predates the CET requirement and is unaffected; Server 2025 ships with it.

The other common cause of a pre-logger crash is a blank `DatabasePath`, which yields the connection string `Data Source=` and opens a private temporary database per connection, so the schema migration fails on a table that cannot exist. Check `settings.json` in the data folder.

Database schema mismatch

The service refuses to start and the log ends with "The existing database was created by an earlier version and its schema does not match this version".

The database predates the introduction of schema migrations and cannot be upgraded automatically. Reset it (adjust the path if you chose a different data folder at install time):

```
Stop-Service DucksInARow
Get-ChildItem "C:\ProgramData\Ducks in a Row\ducks.db*" | Rename-Item -NewName { $_.Name + '.old' }
Start-Service DucksInARow
```

The service recreates the database at the current schema on start. The synced certificate inventory repopulates from the CA on the next sync. ACME accounts, orders, challenge state, and alert history are lost; ACME clients register again automatically on their next run.

Databases created by any version that shipped with migrations upgrade in place automatically; this reset is only ever needed for databases from before that point.

The database journal mode does not match EnableWalMode

The database runs in one of two journal modes. `wal` (write ahead logging) is the default and performs better under concurrent reads and writes, at the cost of two sidecar files, `ducks.db-wal` and `ducks.db-shm`, sitting beside the database while the service runs. `delete` is SQLite's plain rollback journal, and it works on filesystems that WAL cannot use. It keeps no permanent sidecar: a `ducks.db-journal` appears only for the moment a write is in progress and is removed when that write commits, so seeing one briefly is normal.

`Certus:EnableWalMode` chooses between them. Journal mode is stored in the database file rather than in configuration, so the setting and the file can disagree. Every start reports which mode the database is actually in:

```
SQLite journal mode is delete
SQLite journal mode changed from wal to delete
SQLite journal mode is wal after a request for delete; the change was refused ...
```

Read that line rather than inferring the mode from the setting. The first form means nothing needed doing. The second means the setting was applied. The third means it was not.

When the change is refused

Changing journal mode needs the database file exclusively, and the service can only take it at startup, before it begins serving. Anything else holding the file open at that moment refuses the change: a backup agent, an on access scanner, a replication service, a copy of the database open in a SQLite browser, or a second copy of Ducks in a Row pointed at the same data folder.

The service starts normally either way and retries on every start, so a refusal is not urgent. To clear it, stop whatever holds the file and restart the service, then confirm the logged mode.

Moving the data folder to a network share

Do the conversion before the move, not after. A database already in `wal` mode will not open on a filesystem that cannot support WAL, so once it is on the share there is no start on which the service could convert it, and the service fails to start instead:

1. With the data folder still on local disk, set `Certus:EnableWalMode` to `false` in `settings.json` in the data folder.
2. Restart the service and confirm the log reads `SQLite journal mode changed from wal to delete`.
3. Stop the service. Confirm `ducks.db-wal` and `ducks.db-shm` are gone.
4. Move the data folder to the share and update the installer's `DataDirectory` registry value, or set `Certus:DatabasePath` to the new location.
5. Start the service. The log should read `SQLite journal mode is delete`.

Note that WAL is a genuine performance benefit and network shares are a poor place for a SQLite database in general. Prefer local storage for the data folder where you have the choice.

Cannot reach the dashboard, or get a 401

The dashboard and setup API use Windows Integrated Authentication and are limited to the administrator group. Sign in as a member of the built in Administrators group, or set `Auth:AdminGroup` to the Windows or Active Directory group you want to allow, then restart the service.

Still stuck

Ask in [GitHub Discussions](#) or open an [issue](#). For a suspected security problem, follow [SECURITY.md](#) instead.

16. Uninstalling

Remove Ducks in a Row through Add or remove programs, or from the command line:

```
msiexec /x Ducks-in-a-Row.msi
```

If you installed with the setup bundle, uninstall the bundle rather than the MSI, so the entry in Add or remove programs is cleaned up too.

What is removed

- Every application file from `C:\Program Files\Ducks in a Row\`.
- The `DucksInARow` service registration. The service is stopped first.
- The firewall rules the installer created.
- The Start Menu shortcut.

What is kept

- The SQLite database, `ducks.db` in the data folder.
- The log files in `logs\`.
- `settings.json` and `ducks-setup.json`.
- The Data Protection keyring in `keys\`.
- The data folder itself, by default `C:\ProgramData\Ducks in a Row\`.

Keeping the data folder is deliberate. It means an uninstall and reinstall, or an upgrade that goes through one, does not lose your CA configuration, your ACME accounts or your alert history.

The ASP.NET Core runtime is never removed, even if the setup bundle installed it. Other applications on the server may be using it.

The -wal and -shm files are gone, and that is correct

While the service is running you will see `ducks.db-wal` and `ducks.db-shm` beside the database. After an uninstall they are gone.

That is a good sign, not a loss. The database uses write ahead logging, and SQLite writes that log back into `ducks.db` and removes both files when the last connection closes, which is exactly what happens when the uninstaller stops the service. Their absence means every committed change reached `ducks.db`, and that `ducks.db` on its own is now a complete database you can copy or restore.

Removing everything

To remove the data as well, delete the folder by hand after uninstalling:

```
Remove-Item -Recurse -Force "C:\ProgramData\Ducks in a Row"
```

Take a copy first if you might reinstall. The `keys\` folder in there is the Data Protection keyring, and it is the only thing that can decrypt stored EAB secrets and a dashboard saved SMTP password. Deleting it is not recoverable by reinstalling. See [backup and restore](#).

Certificates already issued are not affected

Uninstalling removes the proxy, not the certificates it obtained. Every certificate already issued stays valid until it expires, and stays in the CA's own database. Nothing is revoked.

That cuts both ways: if you are decommissioning the server and want the certificates it obtained to stop working, revoke them **before** you uninstall, while the dashboard can still reach them. See [revocation](#).

17. Glossary

Short, plain explanations of the terms used across these guides.

ACME

A standard protocol (RFC 8555) for requesting and renewing certificates automatically. It is the same protocol Let's Encrypt uses on the public internet. Ducks in a Row speaks ACME on your internal network.

ACME client

A program that requests certificates over ACME, such as certbot, win-acme, Caddy, Traefik, or Posh-ACME. It runs on the machine that needs a certificate.

ADCS (Active Directory Certificate Services)

The Microsoft certificate service built into Windows Server. It runs your internal certificate authority. Ducks in a Row sits in front of ADCS and lets ACME clients enrol from it.

CA (Certificate Authority)

The system that issues and signs certificates. In this product the CA is your ADCS server.

Certificate template

A profile on the CA that decides what a certificate is for and what it may contain, for example a "Web Server" template for TLS server certificates. Ducks in a Row exposes each template as its own ACME endpoint.

ACME directory

The starting URL an ACME client connects to. Ducks in a Row gives each template its own directory URL in the form `https://your-server:5001/acme/<template>/directory`.

Challenge

The check an ACME server runs to confirm you control the name you are asking a certificate for. Ducks in a Row supports three types:

- **HTTP-01:** the server fetches a token over HTTP on port 80.
- **DNS-01:** the server looks up a TXT record in DNS. Good for wildcards.
- **TLS-ALPN-01:** the server checks a certificate over a TLS connection on port 443.

RSAT (Remote Server Administration Tools)

Optional Windows features for managing server roles from another machine. Ducks in a Row needs the ADCS part (RSAT-ADCS-Mgmt) because it registers the COM components used to talk to the CA. Install it with `Install-WindowsFeature RSAT-ADCS-Mgmt`.

DCOM and RPC

The Windows technologies Ducks in a Row uses to call the CA over the network. They need TCP port 135 plus a range of dynamic ports (49152 to 65535) open to the CA.

Machine account

The Active Directory account of the Windows computer itself, written with a trailing \$ (for example DUCKS-SERVER\$). Ducks in a Row acts as this account, so the CA permissions are granted to the machine account, not to a person.

Self signed certificate

A certificate a server generates for itself, with no trusted authority behind it. Ducks in a Row uses one for HTTPS until you give it a certificate your clients already trust. Browsers and ACME clients warn about self signed certificates.

Ducks in a Row | Source available ACME-to-ADCS Proxy
github.com/haruspexsystems/Ducks-in-a-Row | Business Source License 1.1